

🚀 Firecrawl 完整学习手册 ## 从入门到精通 · HawaiiHub实战 ---

📜 官方认证 · 中文版

****包含内容**** ✅ 2,000+ 页官方文档 ✅ 96 个实战项目 ✅ 15 个 HawaiiHub 专项案例 ✅
完整代码示例 ✅ 架构图和流程图 ✅ 最佳实践指南

--- ### ✨ 核心特色 **🚀 实战导向** - 所有案例可直接运行 **📊 可视化** - 20+ 架构图和流程图 **🎯 场景化** - 以夏威夷华人社区为例 **🔧 生产级** - 完整的系统架构设计 **📝 可扩展** - 微服务架构指导

--- ### 👥 适用人群 - 🎓 ****初学者****: 从零开始学习网页爬取 - 🖥️ ****开发者****: 快速集成 Firecrawl API - 🏢 ****创业者****: 构建数据驱动的产品 - 🇮🇹 ****数据分析师****: 自动化数据采集 - 🤖 ****AI工程师****: RAG系统数据源

****版本****: v2.0 ****发布日期****: 2025年10月27日 ****Firecrawl版本****: v2.4.0

****HawaiiHub AI Team**** _Making Web Data LLM-Ready_



版权信息

关于本手册

手册名称: Firecrawl 完整学习手册 - HawaiiHub实战版

版本号: v2.0

发布日期: 2025年10月27日

最后更新: 2025年10月27日

版权声明

内容来源

本手册内容来自以下来源:

1. **Firecrawl 官方文档**

- 来源: <https://docs.firecrawl.dev/>
- 许可: MIT License
- GitHub: <https://github.com/firecrawl/firecrawl> ★ 65k stars

2. **官方示例项目**

- 仓库1: <https://github.com/firecrawl/firecrawl-app-examples> (40个项目)
- 仓库2: <https://github.com/firecrawl/firecrawl/examples> (56个项目)
- 许可: MIT License

3. **HawaiiHub 原创内容**

- 15个实战案例
- 架构设计
- 最佳实践总结
- 版权所有 © 2025 HawaiiHub AI Team

使用许可

本手册采用 知识共享署名-非商业性使用-相同方式共享 4.0 国际许可协议 (CC BY-NC-SA 4.0) 进行许可。

您可以自由地:

-  **分享** - 在任何媒介以任何形式复制、发行本作品
-  **演绎** - 修改、转换或以本作品为基础进行创作

惟须遵守下列条件:

-  **署名** - 您必须给出适当的署名, 提供指向本许可协议的链接
-  **非商业性使用** - 您不得将本作品用于商业目的
-  **相同方式共享** - 如果您修改、转换本作品, 必须使用相同许可协议分发

免责声明

1. 本手册内容仅供学习和参考使用
2. 示例代码和案例数据均为虚构, 不代表真实业务
3. 使用 Firecrawl API 时请遵守目标网站的 robots.txt 和服务条款
4. 作者不对因使用本手册而产生的任何直接或间接损失负责
5. Firecrawl 商标和相关权利属于其各自所有者

商标声明

- **Firecrawl** 是 Mendable AI 的注册商标
- **GPT-4, ChatGPT** 是 OpenAI 的注册商标
- **Gemini** 是 Google 的注册商标
- **Claude** 是 Anthropic 的注册商标
- 其他提及的产品和服务名称可能是其各自所有者的商标

联系方式

项目维护: HawaiiHub AI Team

项目地址: <https://github.com/hawaiihub>

电子邮件: ai@hawaiihub.net

Discord社区: <https://discord.gg/hawaiihub>

贡献者

感谢以下人员对本手册的贡献:

- **技术编写:** HawaiiHub AI Team

- 架构设计: HawaiiHub AI Team
- 案例开发: HawaiiHub AI Team
- 文档翻译: HawaiiHub AI Team
- 技术审校: HawaiiHub AI Team

特别感谢 **Firecrawl 团队** 提供强大的 API 服务和完整的官方文档。

目录

快速导航

本手册共分为 **7 大部分**，包含：

-  15,000+ 字核心教程
 -  2,000+ 页官方文档
 -  96 个实战项目
 -  15 个 HawaiiHub 案例
 -  20+ 架构图和流程图
-

第一部分：手册导读

1. **手册使用指南** (8,000+ 字)
 - 如何使用本手册
 - 文档结构说明
 - 学习建议
 2. **完整学习路线图** (12,000+ 字)
 - 路径1: 快速上手 (1小时)
 - 路径2: 系统学习 (1周)
 - 路径3: 精通进阶 (1个月)
 - 路径4: HawaiiHub实战 (1个月)
 - 路径5: 生产部署 (3个月)
 3. **快速开始指南**
 - 10分钟快速入门
 - 环境配置
 - 第一个示例
-

第二部分：基础入门

1. **Firecrawl 完整学习手册** (15,000+ 字)

- 第1章: Firecrawl 简介
 - 第2章: 核心概念
 - 第3章: 快速开始
 - 第4章: Python SDK 详解
 - 第5章: 核心功能
 - 第6章: 高级特性
 - 第7章: 最佳实践
 - 第8章: HawaiiHub 应用
-

第三部分：官方文档

1. **Firecrawl 官方文档** (2,000+ 页)

- Introduction
 - Features
 - SDKs
 - API Reference
 - Advanced Topics
-

第四部分：API参考

1. 云端 **API** 规范

- Scrape API
- Crawl API
- Map API
- Search API
- Extract API
- Batch Scrape API

2. 完整词汇表

- 网页采集术语
- 网页爬虫术语

- 网页搜索术语
 - 网页提取术语
-

第五部分：实战案例 核心

1. **完整项目总索引** (15,000+ 字)
 - 96个项目完整列表
 - Top 20 推荐项目
 - 技术栈分布
 - AI模型覆盖
 2. **HawaiiHub 实战案例手册** (20,000+ 字)
 - 案例01-05: 数据采集基础
 - 案例06-08: 数据分析
 - 案例09-10: 实时监控
 - 案例11-13: AI应用
 - 案例14-15: 完整系统
 3. **案例架构图集**
 - 20+ Mermaid架构图
 - 系统设计图
 - 数据流程图
 - 性能对比图
-

第六部分：进阶主题

1. **Firecrawl 更新日志** (30,000+ 字)
 - v1.0 - v2.4.0 完整演进
 - 14个月更新历史
 - 重要功能里程碑
-

第七部分：项目集成

1. Cursor 项目集成规范

- FireShot 项目集成
 - 开发环境配置
 - 代码规范
 - 最佳实践
-

附录

A. 配置脚本

- 自动化配置脚本
- 环境验证脚本
- GitHub更新检查脚本

B. 数据示例

- 博客文章数据
- 商家信息数据

C. 工作记录

- 文档整理报告
 - 文档翻译记录
-

总页数: 约 500 页

预计阅读时间: 30-50 小时

实战练习时间: 80-120 小时

****开始您的 Firecrawl 学习之旅! **** 🚀

Firecrawl 学习手册使用指南

版本: v2.0

创建时间: 2025-10-27

适用对象: HawaiiHub 项目团队及所有 Firecrawl 学习者

手册路径: [/Users/zhiledeng/Downloads/FireShot/Firecrawl学习手册/](#)

手册简介

欢迎使用 **Firecrawl 完整学习手册**!

这是一个为 HawaiiHub 项目量身定制的 Firecrawl 综合学习资源库，整合了:

-  官方完整中文文档 (2000+ 页)
 -  实战案例 (40+ 个项目)
 -  配置脚本和工具
 -  API 完整参考
 -  项目集成规范
-

手册结构

目录说明

Firecrawl学习手册/

	📖 00-手册导读/	← 你在这里
	手册使用指南.md	(本文档)
	完整学习路线图.md	(学习路径规划)
	快速开始指南.md	(5 分钟入门)
	快速索引.md	(快速查找)
	🎓 01-基础入门/	
	Firecrawl完整学习手册.md	(15,000+ 字核心教程)
	说明文档.md	(产品说明)
	📖 02-官方文档/	
	官方文档/	(Firecrawl 完整官方文档)
	firecrawl-docs/	(2000+ 页文档, 含中文版)
	官方文档学习计划.md	
	官方文档学习完成报告.md	
	📖 03-API参考/	
	云端API规范.md	
	完整词汇表.md	
	词汇表.md	
	词汇表/	(分类词汇表)
	网页采集API词汇表.md	
	网页爬虫API词汇表.md	
	网页提取API词汇表.md	
	网页搜索API词汇表.md	
	⚙️ 04-配置指南/	
	云端配置指南.md	
	生态系统配置.md	
	配置完成总结.md	
	代码脚本/	(自动化脚本)
	快速配置脚本.sh	
	爬取博客脚本.py	
	分析博客脚本.py	
	🚀 05-实战案例/	
	博客爬取总结.md	
	博客内容.md	

```
|   |— 数据文件/                                (实战数据)
|   |   |— 博客文章数据.json
|   |   |— 博客文章数据.csv
|   |— 示例应用/                                (40+ 个完整项目)
|   |   |— firecrawl-app-examples/
|
|— 🚀 06-进阶主题/
|   |— 生态系统指南.md
|   |— 更新日志.md
|   |— Firecrawl更新日志汇总.md    (v1.0-v2.4.0 完整演进)
|   |— 研究总结.md
|
|— 🔧 07-项目集成/
|   |— Cursor项目集成规范.md        (FireShot 项目专用)
|
|— 📝 工作记录/
|   |— 文档整理报告/
|   |   |— Firecrawl文档最终整理报告.md
|   |— 文档翻译记录/
|   |   |— 文档翻译完成总结.md
|   |   |— 文档翻译状态报告.md
|
|— 文件索引.md                            (完整文件清单)
|— README.md                             (手册总览)
```

🎓 学习路径

路径 1: 快速上手 (1 小时)

目标: 快速掌握基本使用

1. 5 分钟快速开始

bash 📖 阅读: 00-手册导读/快速开始指南.md

1. 30 分钟核心概念

bash 📖 阅读: 01-基础入门/Firecrawl完整学习手册.md 📌 重点: 第 1-3 章

1. 15 分钟配置环境

bash ⚙️ 执行: 04-配置指南/代码脚本/快速配置脚本.sh 📖 参考: 04-配置指南/云端配置指南.md

1. 10 分钟实战

bash 🖥️ 运行: /Users/zhiledeng/Downloads/FireShot/quick_start.py

路径 2: 全面学习 (1 周)

目标: 完整掌握所有功能

Day 1: 基础入门

- ☒ 阅读完整学习手册 (第 1-3 章)
- ☒ 配置开发环境
- ☒ 完成第一个爬取任务

Day 2: API 精通

- ☒ 学习 API 规范
- ☒ 掌握所有端点 (Scrape、Crawl、Map、Search、Extract)
- ☒ 练习 API 调用

Day 3: Python SDK

- ☒ 阅读学习手册第 3 章
- ☒ 掌握 SDK v2 新特性
- ☒ 完成批量爬取实战

Day 4-5: 实战案例

- ☒ 研究 40+ 个示例项目
- ☒ 完成博客爬取案例
- ☒ 自定义 HawaiiHub 应用

Day 6: 高级特性

-  学习 Actions (页面交互)
-  掌握 Change Tracking (变更监控)
-  优化性能和成本

Day 7: 项目集成

-  集成到 HawaiiHub 项目
-  配置 MCP 服务器
-  完成生产环境部署

路径 3: 专家进阶 (持续学习)

目标: 成为 Firecrawl 专家

1. 深入源码

- 研究官方文档 (2000+ 页)
- 分析示例应用源码
- 贡献开源项目

2. 生态系统

- MCP 服务器集成
- LangChain/LlamaIndex 整合
- n8n/Flowise 工作流

3. 前沿技术

- 关注更新日志
 - 测试新功能 (Extract v2、Deep Research)
 - 参与社区讨论
-

快速查找

按需求查找

需求	文档位置
 快速开始	00-手册导读/快速开始指南.md
 完整教程	01-基础入门/Firecrawl完整学习手册.md
 API 文档	03-API参考/云端API规范.md
 配置环境	04-配置指南/云端配置指南.md
 代码示例	05-实战案例/示例应用/
 项目集成	07-项目集成/Cursor项目集成规范.md
 最新更新	06-进阶主题/Firecrawl更新日志汇总.md
 官方文档（中文）	02-官方文档/官方文档/firecrawl-docs/zh/
 词汇表	03-API参考/词汇表/
 自动化脚本	04-配置指南/代码脚本/

按功能查找

功能	相关文档
Scrape（采集）	学习手册第 2.1 章、API 规范、词汇表/网页采集API词汇表.md
Crawl（爬取）	学习手册第 2.2 章、词汇表/网页爬虫API词汇表.md
Map（映射）	学习手册第 2.3 章、官方文档/features/map.mdx
Search（搜索）	学习手册第 2.4 章、词汇表/网页搜索API词汇表.md
Extract（提取）	学习手册第 2.5 章、词汇表/网页提取API词汇表.md
Actions（交互）	学习手册第 4.4 章、官方文档/advanced-scraping-guide.mdx
Batch Scrape	学习手册第 3.3 章、官方文档/features/batch-scrape.mdx
Change Tracking	官方文档/features/change-tracking.mdx

按编程语言查找

语言	文档位置
Python	学习手册第 3 章、官方文档/sdks/python.mdx
JavaScript	官方文档/sdks/node.mdx
Go	官方文档/sdks/go.mdx
Rust	官方文档/sdks/rust.mdx



使用技巧

1. 文档搜索

Mac/Linux:

```
# 搜索关键词
cd /Users/zhiledeng/Downloads/FireShot/Firecrawl学习手册
grep -r "关键词" .

# 搜索 Markdown 文件
find . -name "*.md" -exec grep -l "关键词" {} \;
```

使用 **Cursor AI**:

- 使用 `@workspace` 搜索整个手册
- 使用语义搜索: 询问 "如何使用 Firecrawl 爬取动态网站?"

2. 快速定位

使用快速索引:

```
open "00-手册导读/快速索引.md"
```

使用文件索引:

```
open "文件索引.md"
```

3. 实战练习

推荐顺序:

1. 阅读理论 → 查看示例 → 运行代码 → 修改参数 → 自己实现

示例流程:

```
# 1. 阅读文档
open "01-基础入门/Firecrawl完整学习手册.md"

# 2. 查看示例
cd "05-实战案例/示例应用/firecrawl-app-examples/"
ls -la

# 3. 运行脚本
cd /Users/zhiledeng/Downloads/FireShot
python3 quick_start.py

# 4. 自己实现
# 参考学习手册编写代码
```

4. 问题解决

遇到问题时的查找顺序:

1.  快速索引 ([00-手册导读/快速索引.md](#))
2.  完整学习手册 ([01-基础入门/Firecrawl完整学习手册.md](#))
3.  API 规范 ([03-API参考/云端API规范.md](#))
4.  官方文档 ([02-官方文档/官方文档/firecrawl-docs/zh/](#))
5.  实战案例 ([05-实战案例/](#))
6.  项目集成规范 ([07-项目集成/Cursor项目集成规范.md](#))

手册统计

类别	数量	说明
总文档数	约 3,500+ 个	包括官方文档所有文件
核心教程	1 个 (15K+ 字)	Firecrawl完整学习手册
指南文档	20+ 个	配置、API、实战等
官方文档	2,000+ 页	含中文、英文等多语言
示例项目	40+ 个	完整可运行项目
代码脚本	3 个	自动化配置和爬取脚本
数据文件	2 个	实战数据示例
词汇表	5 个	完整 API 术语参考
工作记录	3 个	文档整理和翻译记录

推荐阅读顺序

新手入门（必读 ★★★★★）

1. 手册使用指南（本文档）- 了解手册结构
2. 快速开始指南 - 5 分钟入门
3. **Firecrawl完整学习手册** - 核心知识体系
4. 云端配置指南 - 环境配置

进阶学习（推荐 ★★★★★）

1. 云端**API**规范 - 深入理解 API
2. 博客爬取总结 - 实战案例学习
3. 生态系统指南 - 了解完整生态

4. 示例应用 - 学习最佳实践

专家级（可选 ★★★）

1. 官方文档（2000+ 页） - 全面深入
2. **Firecrawl**更新日志汇总 - 了解演进
3. 项目集成规范 - HawaiiHub 专用
4. 研究总结 - 前沿技术



配置建议

开发环境

推荐配置:

```
# Python 版本
Python 3.11+

# 必需依赖
pip3 install firecrawl-py python-dotenv requests pydantic

# 可选工具
pip3 install ruff black pytest
```

VS Code/Cursor 扩展:

- Python
- Markdown All in One
- Firecrawl MCP Server（已配置）

环境变量

必需配置:


```
# .env 文件
FIRECRAWL_API_KEY=fc-xxx

# 推荐（密钥轮换）
FIRECRAWL_API_KEY_BACKUP_1=fc-xxx
FIRECRAWL_API_KEY_BACKUP_2=fc-xxx
```

参考:

- [04-配置指南/云端配置指南.md](#)
- [/Users/zhiledeng/Downloads/FireShot/env.template](#)



获取帮助

内部资源

1. 手册内查找

- 使用快速索引
- 搜索相关文档
- 查看实战案例

2. 项目资源

- FireShot 项目文档: [/Users/zhiledeng/Downloads/FireShot/](#)
- 测试脚本: [test_api_keys.py](#)
- 快速开始: [quick_start.py](#)

外部资源

1. 官方渠道

- 文档: <https://docs.firecrawl.dev/>
- GitHub: <https://github.com/mendableai/firecrawl>
- Discord: <https://discord.gg/firecrawl>

2. 社区资源

- 示例应用仓库
- MCP 服务器社区
- LangChain/LlamaIndex 集成文档

开始学习

准备好开始了吗？

推荐下一步：

1. 🚀 快速上手

```
bash open "00-手册导读/快速开始指南.md"
```

1. 📖 完整学习

```
bash open "00-手册导读/完整学习路线图.md"
```

1. 🖥️ 立即实战

```
bash cd /Users/zhiledeng/Downloads/FireShot python3 quick_start.py
```

手册维护

更新记录

日期	版本	更新内容
2025-10-27	v2.0	完整重构手册结构，分类整理
2025-10-27	v1.0	初始版本，整合所有文档

维护者

- 项目：HawaiiHub - FireShot
- 团队：HawaiiHub AI Team
- 联系：查看项目 README.md

祝学习愉快！🎓✨

如有任何问题或建议，欢迎查看手册或联系项目维护者。

Firecrawl 完整学习路线图

版本: v2.0
创建时间: 2025-10-27
适用对象: 所有 Firecrawl 学习者
预计总时长: 1 小时 - 3 个月 (根据学习深度)

学习路径总览

入门级 (1小时)
↓
初级开发者 (1周)
↓
中级开发者 (2周)
↓
高级开发者 (1个月)
↓
专家级 (持续学习)

路径 1: 闪电入门 (1 小时)

目标: 快速上手，能够完成基本爬取任务

适合人群:

-  时间紧迫的开发者
-  需要快速验证可行性
-  只需基础爬取功能

学习计划

时间	任务	文档
0-10 分钟	了解 Firecrawl 核心概念	快速开始指南.md
10-30 分钟	阅读学习手册前 3 章	Firecrawl完整学习手册.md (第 1-3 章)
30-40 分钟	配置开发环境	云端配置指南.md
40-50 分钟	运行第一个示例	quick_start.py
50-60 分钟	修改参数，测试不同网站	自己实践

学习成果

完成后你将能够：

-  理解 Firecrawl 的核心概念
-  配置 Python 开发环境
-  使用 Scrape API 爬取单个网页
-  输出 Markdown 格式内容
-  处理基本错误

验证标准

```
# 能够独立完成以下代码并成功运行
from firecrawl import FirecrawlApp
import os

app = FirecrawlApp(api_key=os.getenv("FIRECRAWL_API_KEY"))
result = app.scrape(
    url="https://example.com",
    formats=["markdown"],
    only_main_content=True
)
print(result.markdown)
```

路径 2: 初级开发者 (1 周)

目标: 掌握核心功能, 能够开发简单应用

适合人群:

- ☒ 有 Python 基础
- ☒ 需要开发实际项目
- ☒ 时间较充裕 (每天 1-2 小时)

Day 1: 基础理论 (2 小时)

上午 (1 小时)

- ☐ 完整阅读学习手册第 1-3 章
- ☐ 理解 Scrape、Crawl、Map、Search、Extract 的区别
- ☐ 了解云端 vs 自部署的差异

下午 (1 小时)

- ☐ 配置开发环境
- ☐ 创建第一个项目
- ☐ 运行官方示例

学习资源:

- [Firecrawl完整学习手册.md](#) (第 1-3 章)
 - [云端配置指南.md](#)
 - [quick_start.py](#)
-

Day 2: Scrape API 精通 (2 小时)

上午 (1 小时)

- ☐ 学习 Scrape API 所有参数
- ☐ 理解 formats 参数 (markdown、html、links 等)
- ☐ 掌握 only_main_content、remove_base64_images 等选项

下午 (1 小时)

- ☐ 实战: 爬取 3 种不同类型网站

- 新闻网站
- 电商网站
- 博客网站
- [] 对比不同参数的效果

学习资源:

- [Firecrawl完整学习手册.md](#) (第 2.1 章)
- [网页采集API词汇表.md](#)
- [云端API规范.md](#) (Scrape 部分)

实战项目:

```
# 爬取夏威夷新闻
url = "https://www.hawaiinewsnow.com/"
result = app.scrape(url, formats=["markdown"], only_main_content=True)

# 爬取本地餐厅信息
url = "https://www.yelp.com/biz/..."
result = app.scrape(url, formats=["markdown", "html"])
```

Day 3: Crawl API 深入 (2 小时)

上午 (1 小时)

- [] 学习 Crawl API 工作原理
- [] 理解 limit、max_depth 参数
- [] 掌握 include_paths、exclude_paths 过滤

下午 (1 小时)

- [] 实战: 爬取一个完整博客
- [] 监控爬取进度
- [] 处理大量数据

学习资源:

- [Firecrawl完整学习手册.md](#) (第 2.2 章)
- [网页爬虫API词汇表.md](#)

实战项目:

```
# 爬取 Firecrawl 博客所有文章
result = app.crawl(
    url="https://www.firecrawl.dev/blog",
    limit=50,
    scrape_options={
        "formats": ["markdown"],
        "only_main_content": True
    }
)
```

Day 4: Map & Search (2 小时)

上午 (1 小时)

- [] 学习 Map API 快速发现 URL
- [] 理解 Search API 搜索 + 爬取一体化
- [] 掌握搜索运算符 (AND、OR、NOT)

下午 (1 小时)

- [] 实战: 使用 Map 发现网站结构
- [] 实战: 使用 Search 搜索夏威夷相关内容
- [] 组合使用 Map + Batch Scrape

学习资源:

- [Firecrawl完整学习手册.md](#) (第 2.3-2.4 章)
- [网页搜索API词汇表.md](#)

实战项目:

```
# 1. 发现所有 URL
urls = app.map(url="https://example.com")

# 2. 批量爬取
results = app.batch_scrape(urls["links"][:20], formats=["markdown"])

# 3. 搜索夏威夷华人餐厅
search_results = app.search(
    query="夏威夷 华人 餐厅",
    limit=10,
    scrape_options={"formats": ["markdown"]}
)
```

Day 5: Extract API (2 小时)

上午 (1 小时)

- [] 学习 Extract API 结构化提取
- [] 理解 prompt 和 schema 参数
- [] 使用 Pydantic 定义数据模型

下午 (1 小时)

- [] 实战: 提取商家信息
- [] 实战: 提取招聘信息
- [] 验证数据完整性

学习资源:

- [Firecrawl完整学习手册.md](#) (第 2.5 章)
- [网页提取API词汇表.md](#)

实战项目:


```
from pydantic import BaseModel

class Restaurant(BaseModel):
    name: str
    address: str
    phone: str
    rating: float

result = app.extract(
    urls=["https://example.com/restaurant"],
    schema=Restaurant.model_json_schema(),
    prompt="提取餐厅的名称、地址、电话和评分"
)
```

Day 6: 错误处理与优化 (2 小时)

上午 (1 小时)

- [] 学习异常处理
- [] 实现重试机制
- [] 配置超时设置

下午 (1 小时)

- [] 使用缓存节省成本
- [] 密钥轮换策略
- [] 成本监控

学习资源:

- [Firecrawl完整学习手册.md](#) (第 6 章)
- [Cursor项目集成规范.md](#)

实战项目:

```
# 完整的错误处理

def safe_scrape(url: str, max_retries: int = 3):
    for attempt in range(max_retries):
        try:
            result = app.scrape(url, formats=["markdown"])
            return result
        except RequestTimeoutError:
            if attempt < max_retries - 1:
                time.sleep(2 ** attempt)
            else:
                logging.error(f"失败: {url}")
                return None
```

Day 7: 综合项目 (3 小时)

任务: 构建一个完整的 HawaiiHub 数据采集系统

要求:

- [] 爬取夏威夷本地新闻 (5 个网站)
- [] 提取商家信息 (20 个商家)
- [] 监控租房价格 (10 个房源)
- [] 保存数据到 JSON 和 CSV
- [] 实现完整的错误处理
- [] 添加日志记录

参考项目:

- `company-data-scraper`
- `automated_price_tracking`
- `博客爬取总结.md`

路径 3: 中级开发者 (2 周)

目标: 掌握高级特性, 能够开发复杂应用

适合人群:

- ☒ 完成初级路径
- ☒ 需要使用高级功能
- ☒ 开发生产级应用

Week 1: 高级特性

Day 8-10: Actions (页面交互)

- [] 学习 Actions 完整语法
- [] 实现点击、滚动、输入等操作
- [] 处理动态加载内容
- [] 截图和 PDF 生成

学习资源:

- [Firecrawl完整学习手册.md](#) (第 4.4 章)
- 官方文档: [features/scrape.mdx](#) (Actions 部分)

实战项目:

```
# 登录后爬取数据
result = app.scrape(
    url="https://example.com/login",
    actions=[
        {"type": "click", "selector": "#login-btn"},
        {"type": "write", "selector": "#username", "text": "user"},
        {"type": "write", "selector": "#password", "text": "pass"},
        {"type": "click", "selector": "#submit"},
        {"type": "wait", "milliseconds": 2000},
        {"type": "screenshot"}
    ]
)
```

Day 11-12: Batch Scrape (批量爬取)

- [] 学习并发爬取策略

- [] 优化性能和成本
- [] 处理大规模数据

学习资源:

- [Firecrawl完整学习手册.md](#) (第 3.3 章)
- 官方文档: [features/batch-scrape.mdx](#)

实战项目:

```
# 批量爬取 100 个商家
urls = [f"https://example.com/business/{i}" for i in range(100)]
results = app.batch_scrape(urls, formats=["markdown"])
```

Day 13-14: Change Tracking (变更监控)

- [] 学习变更检测原理
- [] 配置监控规则
- [] 处理变更通知

学习资源:

- 官方文档: [features/change-tracking.mdx](#)
- 示例项目: [change-detection-tutorial](#)

实战项目:

```
# 监控租房价格变化
result = app.scrape(
    url="https://example.com/listings",
    formats=["change_tracking"]
)
```

Week 2: 生态系统集成

Day 15-16: MCP 服务器集成

- [] 配置 Firecrawl MCP 服务器

- [] 在 Cursor AI 中使用 Firecrawl
- [] 自动化 workflows

学习资源:

- 官方文档: [mcp-server.mdx](#)
 - 示例项目: [mcp-document-reader](#)
-

Day 17-18: LangChain/LlamaIndex 集成

- [] Firecrawl + LangChain RAG
- [] 构建知识库
- [] 智能问答系统

学习资源:

- 官方文档: [integrations/langchain.mdx](#)、[integrations/llamaindex.mdx](#)
 - 示例项目: [deepseek-rag](#)、[local-website-chatbot](#)
-

Day 19-20: n8n/Flowise 工作流

- [] 配置 n8n 节点
- [] 创建自动化流程
- [] 定时任务

学习资源:

- HawaiiHub 文档: [N8N-Firecrawl集成指南.md](#)
-

Day 21: 综合项目

任务: 构建一个完整的 AI 驱动数据平台

- [] 使用 Search API 发现内容
- [] 使用 Crawl API 采集数据
- [] 使用 Extract API 结构化提取
- [] LangChain 构建 RAG 知识库
- [] Streamlit 可视化界面

- ☐ 定时任务自动更新
-

路径 4: 高级开发者（1 个月）

目标: 精通所有功能，能够架构大型系统

Week 1-2: 深入源码

- ☐ 研究官方文档 2,000+ 页
- ☐ 分析 40 个示例项目源码
- ☐ 理解底层实现原理

Week 3: 性能优化

- ☐ 并发优化策略
- ☐ 缓存架构设计
- ☐ 成本控制方案
- ☐ 监控和告警系统

Week 4: 架构设计

- ☐ 设计分布式爬取系统
 - ☐ 实现数据管道
 - ☐ 容错和恢复机制
 - ☐ 生产环境部署
-

路径 5: 专家级（持续学习）

目标: 成为 Firecrawl 领域专家

持续关注

1. 官方更新

- 订阅更新日志
- 测试新功能（Extract v2、Deep Research）
- 参与 Beta 测试

2. 社区贡献

- 回答问题
- 分享经验
- 贡献代码

3. 技术分享

- 撰写博客文章
- 录制教程视频
- 参加技术会议



学习检查清单

入门级（1 小时）

- ☐ 理解 Firecrawl 核心概念
- ☐ 配置开发环境
- ☐ 运行第一个示例
- ☐ 能够独立爬取单个网页

初级（1 周）

- ☐ 掌握 5 大核心 API
- ☐ 理解所有主要参数
- ☐ 完成 10+ 个实战练习
- ☐ 开发第一个实际项目

中级（2 周）

- ☐ 掌握所有高级特性

- ☐ 集成到生态系统
- ☐ 优化性能和成本
- ☐ 开发生产级应用

高级（1 个月）

- ☐ 精通所有功能
- ☐ 架构大型系统
- ☐ 深入理解原理
- ☐ 解决复杂问题

专家级（持续）

- ☐ 关注前沿技术
- ☐ 社区贡献
- ☐ 技术分享
- ☐ 引领创新

HawaiiHub 专项学习路径

Phase 1: 数据采集基础（Week 1）

目标: 能够采集基础数据

- ☐ Day 1-2: 学习 Scrape/Crawl API
- ☐ Day 3-4: 实战采集新闻数据
- ☐ Day 5-6: 实战采集商家数据
- ☐ Day 7: 整合数据到数据库

Phase 2: 高级功能（Week 2-3）

目标: 实现复杂采集需求

- ☐ Week 2: Extract API 结构化提取
- ☐ Week 3: Actions 处理动态页面

- ☐ Week 3: Change Tracking 监控变更

Phase 3: 系统集成 (Week 4)

目标: 完整集成到 HawaiiHub

- ☐ 配置 MCP 服务器
- ☐ 集成到 n8n workflow
- ☐ 自动化数据更新
- ☐ 生产环境部署



学习进度追踪

建议使用

我的学习进度

- ☒ 入门级 (1 小时) - 完成日期: 2025-10-27
- ☐ 初级 (1 周) - Day 1/7
 - ☒ Day 1: 基础理论
 - ☐ Day 2: Scrape API
 - ☐ Day 3: Crawl API
 - ☐ Day 4: Map & Search
 - ☐ Day 5: Extract API
 - ☐ Day 6: 优化
 - ☐ Day 7: 综合项目
- ☐ 中级 (2 周) - 未开始
- ☐ 高级 (1 个月) - 未开始



学习建议

1. 循序渐进

- ☒ 不要跳过基础章节
- ☒ 每个概念都要动手实践
- ☒ 遇到问题及时解决

2. 实战导向

- ☒ 边学边做
- ☒ 每天至少一个实战项目
- ☒ 解决实际业务问题

3. 总结提炼

- ☒ 记录学习笔记
- ☒ 整理常用代码片段
- ☒ 构建个人知识库

4. 社区互动

- ☒ 加入 Discord 社区
 - ☒ 关注 GitHub Issues
 - ☒ 分享学习心得
-



获取帮助

遇到问题？

1. 查阅文档 - 99% 的问题都有答案
2. 运行示例 - 对比示例代码
3. 搜索 **Issues** - 可能别人遇到过

4. 提问社区 - Discord/GitHub Discussions

开始学习

准备好了吗？选择适合你的路径开始学习！

推荐起点:

```
# 1. 阅读本文档（学习路线图）
open "/Users/zhiledeng/Downloads/FireShot/Firecrawl学习手册/00-手册导读/完整学习路线图.md"

# 2. 根据选择的路径，打开对应文档
open "/Users/zhiledeng/Downloads/FireShot/Firecrawl学习手册/00-手册导读/快速开始指南.md"

# 3. 开始实践
cd /Users/zhiledeng/Downloads/FireShot
python3 quick_start.py
```

祝学习愉快，早日成为 **Firecrawl** 专家！ 🚀 ✨

最后更新: 2025-10-27

文档版本: v2.0



Firecrawl 5 分钟快速上手

目标: 从零到运行第一个爬虫

时间: 5-10 分钟

难度: ● 简单

⚡ 超快启动 (3 步)

1 运行自动配置脚本 (2 分钟)

```
cd /Users/zhiledeng/Downloads/FireShot
./quick-setup-firecrawl.sh
```

按提示操作:

- 选择 **y** 安装 Node.js SDK (可选)
- 选择 **y** 覆盖 .env 文件

2 配置 API 密钥 (2 分钟)

```
# 1. 访问 https://www.firecrawl.dev/signin 注册登录
# 2. Dashboard → API Keys → Create New Key
# 3. 复制密钥

# 4. 编辑 .env 文件
open .env

# 5. 替换这一行:
# FIRECRAWL_API_KEY=fc-xxxxxxxxxxxxxxxxxxxxxxxxxxxx
# 改为:
# FIRECRAWL_API_KEY=fc-你的真实密钥
```

3 运行第一个爬虫 (1 分钟)

```
# 测试 API
python3 -c "
from firecrawl import FirecrawlApp
import os
from dotenv import load_dotenv

load_dotenv()
app = FirecrawlApp(api_key=os.getenv('FIRECRAWL_API_KEY'))
result = app.scrape(url='https://example.com', formats=['markdown'])
print('✅ 成功! 爬取了', len(result.markdown), '字符')
"

# 运行完整示例
python3 scripts/scrape_firecrawl_blog.py
```

完成! 你已经成功爬取了第一个网页 🎉

三个核心命令

命令 1: Python SDK (最常用)

```
from firecrawl import FirecrawlApp
import os

app = FirecrawlApp(api_key=os.getenv("FIRECRAWL_API_KEY"))
result = app.scrape(url="https://example.com", formats=["markdown"])
print(result.markdown)
```

命令 2: MCP 工具 (Cursor AI 中)

```
// 在 Cursor 中直接调用
mcp_firecrawl_firecrawl_scrape({
  url: 'https://example.com',
  formats: ['markdown'],
})
```

命令 3: Data Connectors (多源集成)

```
import { createDataConnector } from '@mendable/data-connectors'

const connector = createDataConnector({ provider: 'web-scraper' })
connector.setOptions({ urls: ['https://example.com'], mode: 'single_urls' })
const docs = await connector.getDocuments()
```

常见任务速查

任务 1: 爬取单个网页

```
result = app.scrape(  
    url="https://www.hawaiinewsnow.com/",  
    formats=["markdown"],  
    only_main_content=True  
)
```

任务 2: 批量爬取

```
urls = [  
    "https://www.hawaiinewsnow.com/",  
    "https://www.staradvertiser.com/",  
]  
results = app.batch_scrape(urls=urls, formats=["markdown"])
```

任务 3: 爬取整个网站

```
result = app.crawl(  
    url="https://docs.firecrawl.dev/",  
    limit=100,  
    scrape_options={"formats": ["markdown"]}  
)
```

任务 4: 搜索 + 爬取

```
result = app.search(  
    query="夏威夷 华人 餐厅",  
    limit=10,  
    scrape_options={"formats": ["markdown"]}  
)
```

项目文件速查

文件	用途	何时使用
<code>.env</code>	API 密钥配置	必须配置
<code>quick-setup-firecrawl.sh</code>	自动配置脚本	首次配置时运行
<code>scripts/scrape_firecrawl_blog.py</code>	Python 示例	学习参考
<code>FIRECRAWL_SETUP_COMPLETE.md</code>	配置完成总结	配置后阅读
<code>FIRECRAWL_ECOSYSTEM_SETUP.md</code>	完整指南	深入学习

遇到问题？

问题 1: `FIRECRAWL_API_KEY` 未配置

症状: `{"error": "Invalid API Key"}`

解决:

```
# 检查 .env 文件
cat .env | grep FIRECRAWL_API_KEY

# 如果是 fc-xxxxxxxxxxxxxxxxxxxxxxxxxx, 需要替换为真实密钥
```

问题 2: `firecrawl-py` 未安装

症状: `ModuleNotFoundError: No module named 'firecrawl'`

解决:

```
pip3 install --break-system-packages firecrawl-py python-dotenv
```


问题 3: 脚本没有执行权限

症状: `Permission denied: ./quick-setup-firecrawl.sh`

解决:

```
chmod +x quick-setup-firecrawl.sh
./quick-setup-firecrawl.sh
```

下一步学习路径

初学者（本周）

1.  完成 3 步快速启动
2.  阅读 `FIRECRAWL_SETUP_COMPLETE.md`
3.  修改 `scripts/scrape_firecrawl_blog.py`，爬取不同网站
4.  保存爬取结果到文件

进阶（本月）

1.  学习 Data Connectors（集成 Google Drive、GitHub）
2.  配置 Nango OAuth
3.  实现缓存机制（Redis）
4.  创建数据分析脚本

高级（本季度）

1.  构建 RAG 应用（使用 Firestarter）
2.  集成 AI 研究助手（使用 Open Researcher）
3.  部署自动化爬取系统（使用 GitHub Actions）
4.  监控和成本优化

快速链接

- 获取 **API Key**: <https://www.firecrawl.dev/signin>
 - 完整文档: <https://docs.firecrawl.dev/>
 - **Playground**: <https://www.firecrawl.dev/playground>
 - **GitHub**: <https://github.com/firecrawl/firecrawl>
 - **Discord**: <https://discord.gg/gSmWdAkdwd>
-

检查清单

配置完成后，确认以下项目：

- ☒ 运行了 `quick-setup-firecrawl.sh`
- ☒ `.env` 文件中配置了真实的 `FIRECRAWL_API_KEY`
- ☒ 成功运行了 Python 测试命令
- ☒ 成功运行了 `scripts/scrape_firecrawl_blog.py`
- ☒ 阅读了 `FIRECRAWL_SETUP_COMPLETE.md`

全部完成 = 🎉 你已经准备好开始实战项目了！

版本: v1.0

更新时间: 2025-10-27

维护者: HawaiiHub AI Team

Firecrawl 完整学习手册

创建时间: 2025-10-27

文档来源: 官方中文文档 + 实战经验

适用版本: v2.4.0

目标读者: HawaiiHub 项目团队

目录

- [第一章: Firecrawl 基础](#)
- [第二章: 核心功能详解](#)
- [第三章: Python SDK 完全指南](#)
- [第四章: 高级特性](#)
- [第五章: HawaiiHub 实战应用](#)
- [第六章: 最佳实践与优化](#)
- [附录](#)

第一章: Firecrawl 基础

1.1 什么是 Firecrawl?

Firecrawl 是一个将网站转换为 LLM-ready (适配大语言模型) 数据的开源 API 服务。

核心价值:

-  **零配置:** 无需 sitemap, 自动发现所有页面
-  **LLM-ready:** 输出干净的 Markdown、结构化 JSON
-  **解决棘手问题:** 代理、反爬、JS 渲染、速率限制
-  **极速:** 数秒内返回结果, 支持高吞吐场景
-  **高度可定制:** 排除标签、自定义 headers、设置深度

1.2 开源 vs 云端

功能	开源版	云端版
基础爬取	✓	✓
高级反爬绕过	✗	✓
全球代理池	✗	✓
Actions（页面交互）	✗	✓
Change Tracking（变更监控）	✗	✓
99.9% 可用性	✗	✓
并发优化	有限	✓ 高性能
免费额度	-	500 credits/月

HawaiiHub 选择: ✓ 云端版（高级功能 + 稳定性保证）

1.3 5 大核心功能

1. Scrape（单页爬取）

- 输入：单个 URL
- 输出：Markdown、HTML、JSON、截图、摘要
- 适用：单篇文章、产品页面

2. Crawl（整站爬取）

- 输入：起始 URL
- 输出：所有子页面的内容
- 适用：整站数据采集、文档爬取

3. Map（站点地图）

- 输入：网站 URL
- 输出：所有 URL 列表

- 适用：快速发现网站结构

4. Search（智能搜索）

- 输入：搜索关键词
- 输出：搜索结果 + 完整内容
- 适用：内容发现、竞品监控

5. Extract（数据提取）

- 输入：URL + Schema/Prompt
- 输出：结构化数据
- 适用：表单数据、产品信息

1.4 快速上手（5 分钟）

步骤 1: 安装 SDK

```
pip install firecrawl-py python-dotenv
```

步骤 2: 配置 API Key

```
# .env  
FIRECRAWL_API_KEY=fc-your-key-here
```

步骤 3: 第一个爬取

```
from firecrawl import FirecrawlApp
import os
from dotenv import load_dotenv

load_dotenv()
app = FirecrawlApp(api_key=os.getenv("FIRECRAWL_API_KEY"))

# 爬取单页
result = app.scrape(
    url="https://www.hawaiinewsnow.com/",
    formats=["markdown"],
    only_main_content=True
)

print(result.markdown[:500]) # 前 500 字符
```

预期输出:

✅ 成功返回干净的 Markdown

📄 标题: Hawaii News Now - Breaking News...

📝 内容长度: ~36,000 字符

🕒 耗时: ~1 秒

第二章: 核心功能详解

2.1 Scrape（单页爬取）

基本用法

```
from firecrawl import FirecrawlApp
app = FirecrawlApp(api_key="fc-your-key")

result = app.scrape(
    url="https://example.com",
    formats=["markdown"],           # 输出格式
    only_main_content=True         # 只要主要内容
)

# 访问结果 (SDK v2)
content = result.markdown          # ✅ 属性访问
title = result.metadata.title
url = result.url
```

支持的输出格式

格式	用途	示例
markdown	LLM 训练、RAG	博客文章、新闻
html	保留结构	复杂布局
rawHtml	原始 HTML	调试、分析
screenshot	可视化	页面快照
links	链接提取	SEO 分析
json	结构化数据	产品信息
summary	摘要	快速预览
images	图片 URL	图像采集

多格式同时获取

```
result = app.scrape(  
    url="https://example.com",  
    formats=["markdown", "html", "links", "screenshot"]  
)  
  
# 访问不同格式  
print(result.markdown)    # Markdown 内容  
print(result.html)        # HTML 内容  
print(result.links)       # 链接列表  
print(result.screenshot)  # Base64 编码的截图
```

JSON 模式（结构化提取）

方式 1: 使用 Schema

```
from pydantic import BaseModel  
  
class Article(BaseModel):  
    title: str  
    author: str  
    date: str  
    summary: str  
  
result = app.scrape(  
    url="https://news.example.com/article",  
    formats=[  
        {"type": "json",  
         "schema": Article.model_json_schema()  
        }  
    ]  
)  
  
print(result.json)  # 结构化数据
```


方式 2: 使用 Prompt (无 Schema)

```
result = app.scrape(  
    url="https://restaurant.com",  
    formats=[  
        "type": "json",  
        "prompt": "提取餐厅名称、地址、电话、营业时间"  
    ]  
)  
  
print(result.json) # LLM 自动决定结构
```

Actions (页面交互)

```
result = app.scrape(  
    url="https://google.com",  
    formats=["markdown", "screenshot"],  
    actions=[  
        {"type": "wait", "milliseconds": 2000},  
        {"type": "write", "text": "Firecrawl", "selector": "textarea[name='q']"},  
        {"type": "press", "key": "Enter"},  
        {"type": "wait", "milliseconds": 3000},  
        {"type": "screenshot", "fullPage": False}  
    ]  
)
```

支持的 Actions:

- `wait` - 等待指定时间
- `click` - 点击元素
- `write` - 输入文本
- `press` - 按键
- `scroll` - 滚动页面
- `screenshot` - 截图

2.2 Crawl（整站爬取）

基本用法

```
# 阻塞式（等待完成）
crawl_result = app.crawl(
    url="https://example.com/blog/",
    limit=100,                      # 最多爬取 100 页
    scrape_options={
        "formats": ["markdown"],
        "only_main_content": True
    }
)

print(f"爬取了 {len(crawl_result.data)} 个页面")
for page in crawl_result.data:
    print(f"- {page.url}: {len(page.markdown)} 字符")
```

非阻塞式（启动后轮询）

```
# 启动爬取任务
crawl_job = app.start_crawl(
    url="https://example.com",
    limit=100
)

print(f"任务 ID: {crawl_job.id}")

# 检查状态
status = app.get_crawl_status(crawl_job.id)
print(f"状态: {status.status}")
print(f"已完成: {status.completed}/{status.total}")
```

高级配置

```
crawl_result = app.crawl(  
    url="https://example.com",  
    limit=200,  
    max_depth=3,                # 最大爬取深度  
    allow_subdomains=False,     # 是否包含子域名  
    crawl_entire_domain=False,  # 是否爬取整个域名  
    scrape_options={  
        "formats": ["markdown", {"type": "json", "prompt": "提取标题和日期"}],  
        "only_main_content": True,  
        "exclude_tags": ["nav", "footer", "aside"],  
        "max_age": 3600000,      # 缓存 1 小时  
    }  
)
```

Crawl 参数详解

参数	类型	默认值	说明
<code>url</code>	string	-	起始 URL（必需）
<code>limit</code>	int	10000	最大爬取页面数
<code>max_depth</code>	int	-	最大爬取深度
<code>allow_subdomains</code>	bool	False	是否包含子域名
<code>crawl_entire_domain</code>	bool	False	是否爬取整个域名
<code>scrape_options</code>	object	{}	Scrape 选项
<code>webhook</code>	string	-	Webhook URL

WebSocket 实时监控

```
from firecrawl.utils import create_watcher

# 启动爬取任务
crawl_job = app.start_crawl(
    url="https://example.com",
    limit=50
)

# 创建监控器
watcher = create_watcher(crawl_job.id, app)

# 注册事件处理器
@watcher.on("page")
def on_page(page):
    print(f"✅ 爬取: {page.url} ({len(page.markdown)} 字符)")

@watcher.on("completed")
def on_completed(data):
    print(f"🎉 完成! 共 {len(data)} 页")

@watcher.on("failed")
def on_failed(error):
    print(f"❌ 失败: {error}")

# 开始监控
watcher.start()
```

2.3 Map（站点地图）

基本用法

```
map_result = app.map(  
    url="https://example.com"  
)  
  
print(f"发现 {len(map_result.links)} 个 URL")  
for url in map_result.links[:10]:  
    print(f"- {url}")
```

高级用法

```
map_result = app.map(  
    url="https://example.com",  
    search="blog",           # 只包含 "blog" 的 URL  
    ignore_sitemap=False,    # 使用 sitemap  
    include_subdomains=False, # 不包含子域名  
    limit=5000               # 最多返回 5000 个 URL  
)
```

应用场景:

1. **先 Map 后 Crawl**: 先发现所有 URL，再批量爬取
2. **选择性爬取**: 只爬取特定路径
3. **站点分析**: 了解网站结构

2.4 Search（智能搜索）

基本用法

```
search_result = app.search(
    query="夏威夷 华人 餐厅",
    sources=[{"type": "web"}], # 搜索来源: web、news、images
    limit=10
)

for item in search_result:
    print(f"标题: {item.title}")
    print(f"URL: {item.url}")
```

搜索 + 爬取

```
search_result = app.search(
    query="Firecrawl Python tutorial",
    sources=[{"type": "web"}],
    limit=5,
    scrape_options={
        "formats": ["markdown"],
        "only_main_content": True
    }
)

# 直接获取搜索结果的完整内容
for item in search_result:
    print(f"\n{item.title}")
    print(f"{item.markdown[:200]}...")
```

搜索分类 (v2.1.0+)

```
# GitHub 搜索
github_results = app.search(
    query="firecrawl python",
    categories=["github"] # GitHub 仓库、代码、Issues、文档
)

# 学术搜索
research_results = app.search(
    query="AI machine learning",
    categories=["research"] # arXiv、Nature、IEEE、PubMed
)

# PDF 搜索 (v2.4.0+)
pdf_results = app.search(
    query="研究报告",
    categories=["pdf"]
)
```

2.5 Batch Scrape (批量爬取)

基本用法

```
urls = [  
    "https://example.com/page1",  
    "https://example.com/page2",  
    "https://example.com/page3"  
]  
  
# 阻塞式 (等待完成)  
batch_result = app.batch_scrape(  
    urls=urls,  
    formats=["markdown"],  
    only_main_content=True  
)  
  
for page in batch_result:  
    print(f"{page.url}: {len(page.markdown)} 字符")
```

非阻塞式

```
# 启动批量任务  
batch_job = app.start_batch_scrape(  
    urls=urls,  
    formats=["markdown"]  
)  
  
# 检查状态  
status = app.get_batch_scrape_status(batch_job.id)  
print(f"进度: {status.completed}/{status.total}")
```


3.1 安装与初始化

安装

```
pip install firecrawl-py python-dotenv
```

初始化

```
from firecrawl import FirecrawlApp
import os
from dotenv import load_dotenv

load_dotenv()

# 方式 1: 环境变量
app = FirecrawlApp(api_key=os.getenv("FIRECRAWL_API_KEY"))

# 方式 2: 直接传参
app = FirecrawlApp(api_key="fc-your-key-here")
```

3.2 SDK v2 重要变化

命名约定变化

```
#  v2 正确 (下划线)
result = app.scrape(
    url="...",
    only_main_content=True,      #  下划线
    max_age=172800000,          #  下划线
    block_ads=True,             #  下划线
    skip_tls_verification=True  #  下划线
)

#  v1 错误 (驼峰式)
result = app.scrape(
    url="...",
    onlyMainContent=True,       #  会报错
    maxAge=172800000           #  会报错
)
```

返回值类型变化

```
#  v2 正确 (Document 对象)
result = app.scrape(url="...", formats=["markdown"])

content = result.markdown      #  属性访问
title = result.metadata.title  #  元数据访问
url = result.url               #  URL 访问

#  v1 错误 (字典访问)
content = result.get("markdown") #  报错
content = result["markdown"]     #  报错
```

3.3 完整 API 参考

Scrape 参数

```
result = app.scrape(
    url="https://example.com",          # 必需: URL
    formats=["markdown", "html", "links"], # 输出格式
    only_main_content=True,             # 只要主要内容
    include_tags=["article", "main"],    # 包含的标签
    exclude_tags=["nav", "footer"],      # 排除的标签
    wait_for=5000,                      # 等待时间 (毫秒)
    timeout=30000,                      # 超时时间
    max_age=3600000,                   # 缓存时间 (1小时)
    actions=[...],                     # 页面交互
    location={                          # 地理位置
        "country": "US",
        "languages": ["en-US"]
    },
    mobile=False,                       # 移动端模式
    skip_tls_verification=False,        # 跳过 TLS 验证
    remove_base64_images=True,          # 移除 Base64 图片
    block_ads=True                      # 屏蔽广告
)
```

Crawl 参数

```
crawl_result = app.crawl(
    url="https://example.com",
    limit=100,                          # 最大页面数
    max_depth=3,                        # 最大深度
    allow_subdomains=False,             # 子域名
    crawl_entire_domain=False,          # 整个域名
    ignore_sitemap=False,               # 忽略 sitemap
    include_paths=["/blog/"],           # 包含路径
    exclude_paths=["/admin/"],          # 排除路径
    scrape_options={...}                # Scrape 选项
)
```

3.4 异步支持

```
from firecrawl import AsyncFirecrawlApp
import asyncio

async def main():
    app = AsyncFirecrawlApp(api_key="fc-your-key")

    # 异步爬取
    result = await app.scrape(
        url="https://example.com",
        formats=["markdown"]
    )

    print(result.markdown)

asyncio.run(main())
```

3.5 错误处理

```
from firecrawl import FirecrawlApp
from firecrawl.exceptions import (
    FirecrawlError,
    RequestTimeoutError,
    RateLimitError
)

app = FirecrawlApp(api_key="fc-your-key")

try:
    result = app.scrape(url="https://example.com")
except RequestTimeoutError as e:
    print(f"超时: {e}")
except RateLimitError as e:
    print(f"速率限制: {e}")
except FirecrawlError as e:
    print(f"Firecrawl 错误: {e}")
except Exception as e:
    print(f"未知错误: {e}")
```

第四章：高级特性

4.1 缓存策略

默认缓存（2 天）

```
# 默认 maxAge = 172800000 毫秒（2 天）
result = app.scrape(
    url="https://example.com",
    formats=["markdown"]
)

# 如果 2 天内有缓存，直接返回
```

自定义缓存时间

```
# 10 分钟缓存
result = app.scrape(
    url="https://example.com",
    max_age=600000, # 10 分钟
    formats=["markdown"]
)

# 始终获取最新
result = app.scrape(
    url="https://example.com",
    max_age=0, # 不使用缓存
    formats=["markdown"]
)

# 不存储到缓存
result = app.scrape(
    url="https://example.com",
    store_in_cache=False,
    formats=["markdown"]
)
```

成本优化:

- 默认 2 天缓存可节省 **50%+ 成本**
- 新闻网站: 1 小时缓存
- 静态文档: 7 天缓存
- 实时数据: `max_age=0`

4.2 地理位置与语言

```
result = app.scrape(  
    url="https://example.com",  
    formats=["markdown"],  
    location={  
        "country": "JP",          # 日本  
        "languages": ["ja-JP", "en"] # 日语优先，英语次之  
    }  
)
```

支持的国家代码（部分）：

- **US** - 美国
- **GB** - 英国
- **AU** - 澳大利亚
- **JP** - 日本
- **CN** - 中国
- **DE** - 德国
- **FR** - 法国

4.3 Stealth Mode（隐身模式）

针对高级反爬网站：

```
result = app.scrape(  
    url="https://protected-site.com",  
    formats=["markdown"],  
    proxy="stealth" # 使用隐身代理  
)
```

适用场景：

- 大型电商网站（Amazon、eBay）
- 社交媒体（LinkedIn）
- 受保护的新闻站点

4.4 Change Tracking (变更监控)

```
# 首次爬取
result1 = app.scrape(
    url="https://example.com/product",
    formats=["markdown"]
)

# 定期检查变更
import time
time.sleep(3600) # 1 小时后

result2 = app.scrape(
    url="https://example.com/product",
    formats=["markdown", "changeTracking"]
)

# 检查是否有变更
if result2.change_tracking.changed:
    print("页面已更新! ")
    print(f"变更内容: {result2.change_tracking.diff}")
```


4.5 Extract（智能数据提取）

单页提取

```
from pydantic import BaseModel

class Product(BaseModel):
    name: str
    price: float
    description: str
    rating: float

result = app.scrape(
    url="https://shop.example.com/product",
    formats=[{
        "type": "json",
        "schema": Product.model_json_schema()
    }]
)

product = result.json
print(f"{product['name']}: ${product['price']}")
```

整站提取 (FIRE-1 Beta)

```
extract_result = app.extract(  
    urls=["https://shop.com/category1", "https://shop.com/category2"],  
    prompt="提取所有产品的名称、价格、库存状态",  
    schema={  
        "type": "object",  
        "properties": {  
            "products": {  
                "type": "array",  
                "items": {  
                    "type": "object",  
                    "properties": {  
                        "name": {"type": "string"},  
                        "price": {"type": "number"},  
                        "in_stock": {"type": "boolean"}  
                    }  
                }  
            }  
        }  
    }  
)
```

第五章：HawaiiHub 实战应用

5.1 夏威夷新闻爬取系统

目标网站

1. **Hawaii News Now** - <https://www.hawaiinewsnow.com/>
2. **Star Advertiser** - <https://www.staradvertiser.com/>
3. **Civil Beat** - <https://www.civilbeat.org/>

实现代码

```

from firecrawl import FirecrawlApp
from datetime import datetime
import json
import os
from dotenv import load_dotenv

load_dotenv()
app = FirecrawlApp(api_key=os.getenv("FIRECRAWL_API_KEY"))

class HawaiiNewsScrap:
    """夏威夷新闻爬虫"""

    def __init__(self):
        self.sources = [
            "https://www.hawaiinewsnow.com/",
            "https://www.staradvertiser.com/",
            "https://www.civilbeat.org/"
        ]

    def scrape_homepage(self, url: str) -> dict:
        """爬取新闻首页"""
        try:
            result = app.scrape(
                url=url,
                formats=["markdown", "links"],
                only_main_content=True,
                max_age=3600000 # 1 小时缓存
            )

            return {
                "url": url,
                "success": True,
                "content": result.markdown,
                "links": result.links,
                "scraped_at": datetime.now().isoformat()
            }
        except Exception as e:
            return {
                "url": url,
                "success": False,

```

```

        "error": str(e)
    }

def extract_article_links(self, links: list) -> list:
    """提取文章链接"""
    article_links = []
    for link in links:
        # 过滤条件
        if any(keyword in link.lower() for keyword in ['article', 'news', 'story']):
            if not any(exclude in link.lower() for exclude in ['video', 'weather', 'sports']):
                article_links.append(link)
    return article_links[:10] # 限制 10 篇

def scrape_articles(self, article_links: list) -> list:
    """批量爬取文章"""
    try:
        batch_result = app.batch_scrape(
            urls=article_links,
            formats=["markdown", {"type": "json", "prompt": "提取标题、作者、日期、摘要"}],
            only_main_content=True
        )

        articles = []
        for page in batch_result:
            articles.append({
                "url": page.url,
                "markdown": page.markdown,
                "metadata": page.json if hasattr(page, 'json') else {}
            })

        return articles
    except Exception as e:
        print(f"批量爬取失败: {e}")
        return []

def run(self):
    """执行完整爬取流程"""
    all_articles = []

    for source in self.sources:
        print(f"\n📄 爬取: {source}")

```

```

# 1. 爬取首页
homepage = self.scrape_homepage(source)
if not homepage["success"]:
    print(f"❌失败: {homepage['error']}")
    continue

# 2. 提取文章链接
article_links = self.extract_article_links(homepage["links"])
print(f"📄发现 {len(article_links)} 篇文章")

# 3. 批量爬取文章
articles = self.scrape_articles(article_links)
all_articles.extend(articles)

print(f"✅成功爬取 {len(articles)} 篇文章")

# 保存结果
self.save_results(all_articles)
return all_articles

def save_results(self, articles: list):
    """保存爬取结果"""
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

    # JSON 格式
    with open(f"data/hawaii_news_{timestamp}.json", "w", encoding="utf-8") as f:
        json.dump(articles, f, ensure_ascii=False, indent=2)

    # Markdown 格式
    with open(f"data/hawaii_news_{timestamp}.md", "w", encoding="utf-8") as f:
        f.write(f"# 夏威夷新闻汇总\n\n")
        f.write(f"> 爬取时间: {datetime.now()}\n\n")
        for i, article in enumerate(articles, 1):
            f.write(f"## {i}. {article.get('url', 'N/A')}\n\n")
            f.write(f"{article['markdown'][:500]}...\n\n")
            f.write("---\n\n")

# 使用
scraper = HawaiiNewsScrap()

```

```
articles = scraper.run()  
print(f"\n🎉 完成! 共采集 {len(articles)} 篇文章")
```

5.2 与 NewsAPI 集成


```

from firecrawl import FirecrawlApp
from newsapi import NewsApiClient
import os

# 初始化
firecrawl = FirecrawlApp(api_key=os.getenv("FIRECRAWL_API_KEY"))
newsapi = NewsApiClient(api_key=os.getenv("NEWSAPI_KEY"))

def fetch_hawaii_news():
    """NewsAPI 发现 + Firecrawl 采集"""

    # 1. 使用 NewsAPI 搜索
    articles = newsapi.get_everything(
        q="Hawaii Chinese community",
        language="en",
        sort_by="publishedAt",
        page_size=20
    )

    # 2. 提取 URL
    urls = [article["url"] for article in articles["articles"]]

    # 3. 使用 Firecrawl 批量爬取完整内容
    full_articles = firecrawl.batch_scrape(
        urls=urls,
        formats=["markdown"],
        only_main_content=True
    )

    # 4. 合并数据
    results = []
    for i, full_article in enumerate(full_articles):
        results.append({
            **articles["articles"][i], # NewsAPI 元数据
            "full_content": full_article.markdown # Firecrawl 完整内容
        })

    return results

```

5.3 成本监控与优化

```

class CostMonitor:
    """成本监控器"""

    def __init__(self, daily_budget: float = 10.0):
        self.daily_budget = daily_budget
        self.request_count = 0
        self.total_cost = 0.0

    def track_request(self, cost: float = 0.01):
        """记录请求成本"""
        self.request_count += 1
        self.total_cost += cost

        if self.total_cost >= self.daily_budget:
            raise Exception(f"超出每日预算: ${self.daily_budget}")

        print(f"请求 #{self.request_count} | 成本: ${cost:.4f} | 累计: ${self.total_cost:.2f}/{self.daily_budget}")

    def get_stats(self) -> dict:
        """获取统计信息"""
        return {
            "total_requests": self.request_count,
            "total_cost": self.total_cost,
            "average_cost": self.total_cost / self.request_count if self.request_count > 0 else 0,
            "budget_remaining": self.daily_budget - self.total_cost
        }

# 使用
monitor = CostMonitor(daily_budget=10.0)

def scrape_with_monitoring(url: str) -> dict:
    """带成本监控的爬取"""
    monitor.track_request(cost=0.01)
    result = app.scrape(url=url, formats=["markdown"])
    return result

# 统计信息
stats = monitor.get_stats()
print(f"📊 今日统计: {stats}")

```

第六章：最佳实践与优化

6.1 性能优化

1. 使用缓存

```
# ❌ 错误：每次都重新爬取
for i in range(10):
    result = app.scrape(url="https://example.com", max_age=0)

# ✅ 正确：使用缓存
for i in range(10):
    result = app.scrape(url="https://example.com", max_age=3600000) # 1 小时
```

2. 批量优于单个

```
# ❌ 错误：逐个爬取
urls = ["url1", "url2", "url3"]
for url in urls:
    result = app.scrape(url)

# ✅ 正确：批量爬取
results = app.batch_scrape(urls)
```

3. 只要主要内容

```
# ✅ 正确：移除导航、广告、页脚
result = app.scrape(
    url="...",
    only_main_content=True,
    exclude_tags=["nav", "footer", "aside", "script"],
    block_ads=True
)
```

6.2 错误处理与重试

```
import time

def safe_scrape(url: str, max_retries: int = 3) -> dict | None:
    """安全爬取，带重试"""
    for attempt in range(max_retries):
        try:
            result = app.scrape(
                url=url,
                formats=["markdown"],
                only_main_content=True
            )

            if not result or not hasattr(result, "markdown"):
                raise ValueError("无效结果")

            print(f"✅ 成功爬取: {url}")
            return result

        except RequestTimeoutError as e:
            if attempt < max_retries - 1:
                wait_time = 2 ** attempt # 指数退避
                print(f"⌚ 超时, {wait_time}秒后重试... ({attempt+1}/{max_retries})")
                time.sleep(wait_time)
            else:
                print(f"❌ 失败 ({max_retries}次重试后): {url}")
                return None

        except Exception as e:
            print(f"❌ 错误: {url} - {e}")
            return None
```

6.3 数据验证

```
from pydantic import BaseModel, HttpUrl, Field
from typing import Optional

class Article(BaseModel):
    """文章数据模型"""
    title: str = Field(..., min_length=1, max_length=200)
    url: HttpUrl
    author: str
    date: str
    content: Optional[str] = None

# 验证
try:
    article = Article(
        title="测试文章",
        url="https://example.com",
        author="张三",
        date="2025-10-27",
        content="..."
    )
except ValidationError as e:
    print(f"验证失败: {e}")
```

6.4 日志记录

```
import logging

# 配置日志
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
    handlers=[
        logging.FileHandler("logs/firecrawl.log"),
        logging.StreamHandler()
    ]
)

logger = logging.getLogger("HawaiiNews")

# 使用日志
def scrape_with_logging(url: str):
    logger.info(f"开始爬取: {url}")
    try:
        result = app.scrape(url=url)
        logger.info(f"成功: {len(result.markdown)} 字符")
        return result
    except Exception as e:
        logger.error(f"失败: {url} - {e}")
        return None
```

6.5 数据清洗

```
import re

def clean_markdown(content: str) -> str:
    """清洗Markdown 内容"""
    # 移除多余空行
    content = re.sub(r"\n{3,}", "\n\n", content)

    # 移除特定模式
    patterns = [
        r"Subscribe to.*newsletter",
        r"Advertisement",
        r"Related Articles",
        r"Share on.*"
    ]

    for pattern in patterns:
        content = re.sub(pattern, "", content, flags=re.IGNORECASE)

    return content.strip()
```

附录

A. 术语表

术语	中文	说明
Scrape	爬取/抓取	单页内容采集
Crawl	整站爬取	递归爬取所有子页面
Map	站点地图	快速发现所有 URL
Extract	提取	结构化数据提取
LLM-ready	适配 LLM	适合大语言模型使用的格式
Schema	架构	数据结构定义
Actions	操作/动作	页面交互（点击、输入等）
Stealth Mode	隐身模式	绕过高级反爬机制

B. 错误码对照

错误码	说明	解决方案
401	API Key 无效	检查环境变量
429	速率限制	降低请求频率或升级计划
500	服务器错误	稍后重试
504	超时	增加 timeout 参数

C. 速查表

常用参数

```
# Scrape
result = app.scrape(
    url="...",
    formats=["markdown"],           # 输出格式
    only_main_content=True,         # 主要内容
    max_age=3600000,               # 缓存 1 小时
    exclude_tags=["nav"]           # 排除标签
)

# Crawl
crawl_result = app.crawl(
    url="...",
    limit=100,                      # 最多 100 页
    max_depth=3,                   # 深度 3 层
    scrape_options={...}           # Scrape 选项
)

# Map
map_result = app.map(
    url="...",
    limit=5000                      # 最多 5000 URL
)

# Search
search_result = app.search(
    query="...",
    sources=[{"type": "web"}],
    limit=10
)
```

D. 资源链接

官方资源

- 📖 在线文档: <https://docs.firecrawl.dev/>
- 🐙 GitHub: <https://github.com/firecrawl/firecrawl>
- 💬 Discord: <https://discord.gg/firecrawl>
- 📝 Changelog: <https://www.firecrawl.dev/changelog>
- 🎮 Playground: <https://firecrawl.dev/playground>

本地资源

- 📁 官方中文文档: `firecrawl-docs/zh/`
- 📖 项目规则: `.cursorrules`
- 🇬🇧 更新日志: `Firecrawl更新日志汇总.md`
- ⚙️ SDK 配置: `SDK_CONFIGURATION_COMPLETE.md`

最后更新: 2025-10-27

文档版本: v1.0

作者: HawaiiHub AI Team

总字数: 约 15,000 字

🎉 恭喜完成学习！现在你已经掌握 **Firecrawl** 的所有核心功能！

Firecrawl 云 API 使用规范

专为 HawaiiHub 团队打造的 Firecrawl 云 API 最佳实践

版本: v1.0.0

更新: 2025-10-27

目录

1. 核心原则
2. [API 密钥管理](#)
3. [请求优化策略](#)
4. [成本控制](#)
5. [错误处理](#)
6. [性能优化](#)
7. [安全规范](#)
8. [监控与日志](#)
9. [代码规范](#)
10. [应急预案](#)

核心原则

1. 最小化请求原则

```
# ✅ 好的做法 - 使用 batch_scrape
urls = [url1, url2, url3]
results = app.batch_scrape(urls, formats=['markdown'])

# ❌ 避免 - 循环调用
for url in urls:
    result = app.scrape(url) # 浪费 API 配额
```

2. 缓存优先原则

```
# ✅ 使用 maxAge 参数启用缓存
result = app.scrape(
    url="https://example.com",
    formats=['markdown'],
    maxAge=3600000 # 1 小时缓存
)
```

3. 内容过滤原则

```
# ✅ 只提取需要的内容
result = app.scrape(
    url="https://example.com",
    formats=['markdown'],
    onlyMainContent=True, # 过滤广告、导航栏等
    removeTags=['script', 'style', 'iframe']
)
```

API 密钥管理

环境变量配置

开发环境 (`.env.development`)

```
# Firecrawl API 配置
FIRECRAWL_API_KEY=fc-dev-xxxxxxxxxxxxxxxx
FIRECRAWL_API_URL=https://api.firecrawl.dev
FIRECRAWL_TIMEOUT=30000
FIRECRAWL_MAX_RETRIES=3

# 功能开关
FIRECRAWL_ENABLE_CACHE=true
FIRECRAWL_CACHE_TTL=3600
FIRECRAWL_ENABLE_DEBUG=true
```

生产环境 (`.env.production`)

```
# Firecrawl API 配置
FIRECRAWL_API_KEY=fc-prod-xxxxxxxxxxxxxxxx
FIRECRAWL_API_URL=https://api.firecrawl.dev
FIRECRAWL_TIMEOUT=60000
FIRECRAWL_MAX_RETRIES=5

# 功能开关
FIRECRAWL_ENABLE_CACHE=true
FIRECRAWL_CACHE_TTL=7200
FIRECRAWL_ENABLE_DEBUG=false

# 速率限制
FIRECRAWL_RATE_LIMIT=100 # 每分钟最大请求数
FIRECRAWL_BURST_LIMIT=20 # 突发流量限制
```

密钥轮换策略

```
# config/firecrawl.py
import os
from datetime import datetime, timedelta

class FirecrawlConfig:
    """Firecrawl 配置管理"""

    # API 密钥（支持多密钥轮换）
    API_KEYS = [
        os.getenv('FIRECRAWL_API_KEY_1'),
        os.getenv('FIRECRAWL_API_KEY_2'), # 备用密钥
        os.getenv('FIRECRAWL_API_KEY_3'), # 备用密钥
    ]

    # 当前使用的密钥索引
    CURRENT_KEY_INDEX = 0

    # 密钥轮换时间（每 24 小时）
    KEY_ROTATION_INTERVAL = timedelta(hours=24)
    LAST_ROTATION = datetime.now()

    @classmethod
    def get_api_key(cls) -> str:
        """获取当前 API 密钥"""

        # 检查是否需要轮换
        if datetime.now() - cls.LAST_ROTATION > cls.KEY_ROTATION_INTERVAL:
            cls.rotate_key()

        return cls.API_KEYS[cls.CURRENT_KEY_INDEX]

    @classmethod
    def rotate_key(cls):
        """轮换到下一个密钥"""

        cls.CURRENT_KEY_INDEX = (cls.CURRENT_KEY_INDEX + 1) % len(cls.API_KEYS)
        cls.LAST_ROTATION = datetime.now()
        print(f"已轮换到密钥 #{cls.CURRENT_KEY_INDEX + 1}")
```

密钥安全规范

✅ 必须遵守

1. 绝不将 API 密钥硬编码到代码中
2. 绝不将 `.env` 文件提交到 Git
3. 必须使用环境变量或密钥管理服务
4. 必须定期轮换密钥（建议每月）
5. 必须为不同环境使用不同密钥

❌ 严禁行为

```
# ❌ 严禁硬编码
app = FirecrawlApp(api_key="fc-xxxxxxxxxxxxxxxx")

# ❌ 严禁打印密钥
print(f"API Key: {api_key}")

# ❌ 严禁在日志中记录密钥
logger.info(f"Using key: {api_key}")
```


✅ 正确做法

```
# ✅ 使用环境变量
import os
from firecrawl import FirecrawlApp

api_key = os.getenv('FIRECRAWL_API_KEY')
if not api_key:
    raise ValueError("未设置 FIRECRAWL_API_KEY 环境变量")

app = FirecrawlApp(api_key=api_key)

# ✅ 日志中隐藏密钥
def mask_api_key(key: str) -> str:
    """隐藏 API 密钥"""
    return f"{key[:8]}...{key[-4:]}" if len(key) > 12 else "***"

logger.info(f"使用 API 密钥: {mask_api_key(api_key)}")
```

请求优化策略

1. 批量处理优化

```

from typing import List, Dict
import asyncio
from firecrawl import FirecrawlApp

class FirecrawlOptimizer:
    """Firecrawl 请求优化器"""

    def __init__(self, api_key: str, batch_size: int = 10):
        self.app = FirecrawlApp(api_key=api_key)
        self.batch_size = batch_size

    def batch_scrape_optimized(
        self,
        urls: List[str],
        formats: List[str] = ['markdown'],
        **kwargs
    ) -> List[Dict]:
        """
        优化的批量爬取

        特性：
            1. 自动分批处理
            2. 并发控制
            3. 失败重试
            4. 进度追踪
        """
        results = []
        total_batches = (len(urls) + self.batch_size - 1) // self.batch_size

        for i in range(0, len(urls), self.batch_size):
            batch = urls[i:i + self.batch_size]
            batch_num = i // self.batch_size + 1

            print(f"处理批次 {batch_num}/{total_batches} ({len(batch)} 个 URL)")

            try:
                batch_results = self.app.batch_scrape(
                    urls=batch,
                    formats=formats,
                    **kwargs

```

```
)
results.extend(batch_results)

except Exception as e:
    print(f"批次 {batch_num} 失败: {e}")
    # 失败重试 (逐个处理)
    for url in batch:
        try:
            result = self.app.scrape(url, formats=formats, **kwargs)
            results.append(result)
        except Exception as url_error:
            print(f"URL {url} 失败: {url_error}")
            results.append({'url': url, 'error': str(url_error)})

return results
```

2. 智能重试策略

```

import time
from typing import Callable, Any
from functools import wraps

def retry_with_exponential_backoff(
    max_retries: int = 3,
    base_delay: float = 1.0,
    max_delay: float = 60.0,
    exponential_base: float = 2.0
):
    """
    指数退避重试装饰器

    参数:
        max_retries: 最大重试次数
        base_delay: 基础延迟 (秒)
        max_delay: 最大延迟 (秒)
        exponential_base: 指数基数
    """

    def decorator(func: Callable) -> Callable:
        @wraps(func)
        def wrapper(*args, **kwargs) -> Any:
            retries = 0
            while retries < max_retries:
                try:
                    return func(*args, **kwargs)

                except Exception as e:
                    retries += 1

                    if retries >= max_retries:
                        raise

            # 计算延迟时间
            delay = min(
                base_delay * (exponential_base ** retries),
                max_delay
            )

            print(f"请求失败, {delay:.1f}秒后重试 ({retries}/{max_retries}): {e}")

```

```
        time.sleep(delay)

        raise Exception(f"超过最大重试次数 ({max_retries})")

    return wrapper
return decorator

# 使用示例
@retry_with_exponential_backoff(max_retries=5)
def scrape_with_retry(url: str):
    """带重试的爬取"""
    return app.scrape(url, formats=['markdown'])
```

3. 并发控制


```

import asyncio
from concurrent.futures import ThreadPoolExecutor
from typing import List, Dict

class ConcurrentScraper:
    """并发爬取管理器"""

    def __init__(
        self,
        api_key: str,
        max_workers: int = 5,
        rate_limit: int = 10 # 每秒最大请求数
    ):
        self.app = FirecrawlApp(api_key=api_key)
        self.max_workers = max_workers
        self.rate_limit = rate_limit
        self.semaphore = asyncio.Semaphore(max_workers)

    async def scrape_url(self, url: str, **kwargs) -> Dict:
        """异步爬取单个 URL"""
        async with self.semaphore:
            try:
                # 速率限制
                await asyncio.sleep(1 / self.rate_limit)

                # 执行爬取
                result = await asyncio.to_thread(
                    self.app.scrape,
                    url,
                    **kwargs
                )
                return {'url': url, 'success': True, 'data': result}

            except Exception as e:
                return {'url': url, 'success': False, 'error': str(e)}

    async def scrape_all(self, urls: List[str], **kwargs) -> List[Dict]:
        """异步爬取所有 URL"""
        tasks = [self.scrape_url(url, **kwargs) for url in urls]
        return await asyncio.gather(*tasks)

```

```
# 使用示例

async def main():
    scraper = ConcurrentScraper(
        api_key=os.getenv('FIRECRAWL_API_KEY'),
        max_workers=5,
        rate_limit=10
    )

    urls = ["https://example.com/1", "https://example.com/2", ...]
    results = await scraper.scrape_all(urls, formats=['markdown'])

    print(f"成功: {sum(1 for r in results if r['success'])}/{len(results)}")

# asyncio.run(main())
```

成本控制

1. 成本监控

```

from datetime import datetime
from typing import Dict, List

class CostTracker:
    """API 成本追踪器"""

    # 定价（根据实际情况调整）
    PRICING = {
        'scrape': 0.005,      # 每页 $0.005
        'crawl': 0.004,      # 每页 $0.004（批量折扣）
        'search': 0.01,      # 每次搜索 $0.01
        'extract': 0.008,    # 每次提取 $0.008
    }

    def __init__(self):
        self.usage = {
            'scrape': 0,
            'crawl': 0,
            'search': 0,
            'extract': 0,
        }
        self.start_time = datetime.now()

    def track(self, operation: str, count: int = 1):
        """记录 API 使用"""
        if operation in self.usage:
            self.usage[operation] += count

    def get_cost(self) -> float:
        """计算总成本"""
        total = sum(
            self.usage[op] * self.PRICING[op]
            for op in self.usage
        )
        return round(total, 2)

    def get_report(self) -> Dict:
        """生成成本报告"""
        runtime = (datetime.now() - self.start_time).total_seconds()

```

```

    return {
        'total_cost': self.get_cost(),
        'usage': self.usage,
        'runtime_seconds': runtime,
        'cost_per_hour': round(self.get_cost() / (runtime / 3600), 2) if runtime > 0 else 0,
        'details': {
            op: {
                'count': self.usage[op],
                'cost': round(self.usage[op] * self.PRICING[op], 2)
            }
            for op in self.usage
        }
    }

# 全局成本追踪器
cost_tracker = CostTracker()

# 装饰器
def track_cost(operation: str):
    """成本追踪装饰器"""
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            result = func(*args, **kwargs)

            # 根据结果追踪成本
            if isinstance(result, list):
                cost_tracker.track(operation, len(result))
            else:
                cost_tracker.track(operation, 1)

            return result
        return wrapper
    return decorator

# 使用示例
@track_cost('scrape')
def scrape_urls(urls: List[str]):
    return app.batch_scrape(urls)

# 定期打印成本报告

```

```
import atexit

def print_cost_report():
    report = cost_tracker.get_report()
    print("\n" + "=" * 50)
    print("📊 Firecrawl API 成本报告")
    print("=" * 50)
    print(f"总成本: ${report['total_cost']}")
    print(f"运行时长: {report['runtime_seconds']:.0f} 秒")
    print(f"每小时成本: ${report['cost_per_hour']}")
    print("\n详细使用:")
    for op, details in report['details'].items():
        print(f"  {op}: {details['count']}次 (${details['cost']})")
    print("=" * 50)

atexit.register(print_cost_report)
```

2. 预算限制

```

class BudgetLimiter:
    """预算限制器"""

    def __init__(
        self,
        daily_budget: float = 10.0,    # 每日预算 $10
        monthly_budget: float = 200.0  # 每月预算 $200
    ):
        self.daily_budget = daily_budget
        self.monthly_budget = monthly_budget
        self.daily_spent = 0.0
        self.monthly_spent = 0.0
        self.last_reset = datetime.now()

    def check_budget(self, estimated_cost: float) -> bool:
        """检查是否超预算"""
        # 重置每日预算
        if (datetime.now() - self.last_reset).days >= 1:
            self.daily_spent = 0.0
            self.last_reset = datetime.now()

        # 检查预算
        if self.daily_spent + estimated_cost > self.daily_budget:
            raise BudgetExceededError(f"超过每日预算 ({self.daily_budget})")

        if self.monthly_spent + estimated_cost > self.monthly_budget:
            raise BudgetExceededError(f"超过每月预算 ({self.monthly_budget})")

        return True

    def record_cost(self, cost: float):
        """记录消费"""
        self.daily_spent += cost
        self.monthly_spent += cost

class BudgetExceededError(Exception):
    """预算超支异常"""
    pass

# 使用示例

```



```
budget = BudgetLimiter(daily_budget=10.0)

def scrape_with_budget(urls: List[str]):
    """带预算控制的爬取"""
    estimated_cost = len(urls) * 0.005

    # 检查预算
    budget.check_budget(estimated_cost)

    # 执行爬取
    results = app.batch_scrape(urls)

    # 记录成本
    budget.record_cost(estimated_cost)

    return results
```

3. 成本优化建议

策略 1: 智能缓存

```

import hashlib
import json
from typing import Optional

class SmartCache:
    """智能缓存管理器"""

    def __init__(self, cache_dir: str = "./cache"):
        self.cache_dir = cache_dir
        os.makedirs(cache_dir, exist_ok=True)

    def _get_cache_key(self, url: str, params: dict) -> str:
        """生成缓存键"""
        data = f"{url}:{json.dumps(params, sort_keys=True)}"
        return hashlib.md5(data.encode()).hexdigest()

    def get(self, url: str, params: dict, max_age: int = 3600) -> Optional[dict]:
        """获取缓存"""
        cache_key = self._get_cache_key(url, params)
        cache_file = os.path.join(self.cache_dir, f"{cache_key}.json")

        if not os.path.exists(cache_file):
            return None

        # 检查缓存是否过期
        mtime = os.path.getmtime(cache_file)
        if time.time() - mtime > max_age:
            os.remove(cache_file)
            return None

        with open(cache_file, 'r') as f:
            return json.load(f)

    def set(self, url: str, params: dict, data: dict):
        """设置缓存"""
        cache_key = self._get_cache_key(url, params)
        cache_file = os.path.join(self.cache_dir, f"{cache_key}.json")

        with open(cache_file, 'w') as f:
            json.dump(data, f)

```

```
# 使用缓存
cache = SmartCache()

def scrape_with_cache(url: str, **params):
    """带缓存的爬取"""
    # 尝试从缓存获取
    cached = cache.get(url, params, max_age=3600)
    if cached:
        print(f"✅ 命中缓存: {url}")
        return cached

    # 缓存未命中, 调用 API
    print(f"🔄 API 请求: {url}")
    result = app.scrape(url, **params)

    # 保存到缓存
    cache.set(url, params, result)

    return result
```

策略 2: 内容去重

```

from typing import Set

class URLDeduplicator:
    """URL 去重器"""

    def __init__(self):
        self.seen_urls: Set[str] = set()
        self.seen_content_hashes: Set[str] = set()

    def is_duplicate_url(self, url: str) -> bool:
        """检查 URL 是否重复"""
        normalized_url = self._normalize_url(url)
        if normalized_url in self.seen_urls:
            return True
        self.seen_urls.add(normalized_url)
        return False

    def is_duplicate_content(self, content: str) -> bool:
        """检查内容是否重复"""
        content_hash = hashlib.md5(content.encode()).hexdigest()
        if content_hash in self.seen_content_hashes:
            return True
        self.seen_content_hashes.add(content_hash)
        return False

    @staticmethod
    def _normalize_url(url: str) -> str:
        """规范化 URL"""
        # 移除查询参数、锚点等
        from urllib.parse import urlparse
        parsed = urlparse(url)
        return f"{parsed.scheme}://{parsed.netloc}{parsed.path}"

# 使用去重
deduplicator = URLDeduplicator()

def scrape_unique_urls(urls: List[str]):
    """只爬取不重复的 URL"""
    unique_urls = [
        url for url in urls

```

```
        if not deduplicator.is_duplicate_url(url)
    ]

    print(f"去重后: {len(unique_urls)}/{len(urls)} 个 URL")
    return app.batch_scrape(unique_urls)
```

错误处理

1. 完整的错误处理体系


```

from enum import Enum
from typing import Optional, Dict, Any

class FirecrawlErrorType(Enum):
    """错误类型枚举"""
    AUTHENTICATION = "认证失败"
    RATE_LIMIT = "速率限制"
    TIMEOUT = "请求超时"
    INVALID_URL = "无效 URL"
    NETWORK = "网络错误"
    SERVER_ERROR = "服务器错误"
    UNKNOWN = "未知错误"

class FirecrawlError(Exception):
    """Firecrawl 自定义异常"""

    def __init__(
        self,
        error_type: FirecrawlErrorType,
        message: str,
        original_error: Optional[Exception] = None,
        context: Optional[Dict[str, Any]] = None
    ):
        self.error_type = error_type
        self.message = message
        self.original_error = original_error
        self.context = context or {}

        super().__init__(self.message)

    def __str__(self):
        return f"[{self.error_type.value}] {self.message}"

    def to_dict(self) -> Dict:
        """转换为字典"""
        return {
            'error_type': self.error_type.name,
            'message': self.message,
            'context': self.context,
            'original_error': str(self.original_error) if self.original_error else None

```

```

    }

class ErrorHandler:
    """统一错误处理器"""

    @staticmethod
    def handle_error(e: Exception, url: Optional[str] = None) -> FirecrawlError:
        """处理并分类错误"""
        error_msg = str(e).lower()
        context = {'url': url} if url else {}

        # 认证错误
        if 'unauthorized' in error_msg or 'api key' in error_msg:
            return FirecrawlError(
                FirecrawlErrorType.AUTHENTICATION,
                "API 密钥无效或已过期",
                e,
                context
            )

        # 速率限制
        if 'rate limit' in error_msg or '429' in error_msg:
            return FirecrawlError(
                FirecrawlErrorType.RATE_LIMIT,
                "超过 API 速率限制，请稍后重试",
                e,
                context
            )

        # 超时
        if 'timeout' in error_msg:
            return FirecrawlError(
                FirecrawlErrorType.TIMEOUT,
                "请求超时",
                e,
                context
            )

        # 无效 URL
        if 'invalid url' in error_msg:
            return FirecrawlError(

```

```

        FirecrawlErrorType.INVALID_URL,
        f"无效的 URL: {url}",
        e,
        context
    )

# 网络错误
if 'connection' in error_msg or 'network' in error_msg:
    return FirecrawlError(
        FirecrawlErrorType.NETWORK,
        "网络连接失败",
        e,
        context
    )

# 服务器错误
if '500' in error_msg or '502' in error_msg or '503' in error_msg:
    return FirecrawlError(
        FirecrawlErrorType.SERVER_ERROR,
        "Firecrawl 服务器错误",
        e,
        context
    )

# 未知错误
return FirecrawlError(
    FirecrawlErrorType.UNKNOWN,
    f"未知错误: {str(e)}",
    e,
    context
)

# 使用示例
def safe_scrape(url: str, **kwargs) -> Dict:
    """安全的爬取（带错误处理）"""
    try:
        return app.scrape(url, **kwargs)

    except Exception as e:
        # 统一错误处理
        error = ErrorHandler.handle_error(e, url)

```

```
# 记录错误日志
logger.error(f"爬取失败: {error}", extra=error.to_dict())

# 根据错误类型决定是否重试
if error.error_type == FirecrawlErrorType.RATE_LIMIT:
    time.sleep(60) # 等待 1 分钟后重试
    return safe_scrape(url, **kwargs)

elif error.error_type == FirecrawlErrorType.TIMEOUT:
    # 增加超时时间重试
    kwargs['timeout'] = kwargs.get('timeout', 30) * 2
    return safe_scrape(url, **kwargs)

# 其他错误直接抛出
raise error
```

2. 错误日志记录

```

import logging
from logging.handlers import RotatingFileHandler

# 配置日志
def setup_logging():
    """配置 Firecrawl 日志"""
    logger = logging.getLogger('firecrawl')
    logger.setLevel(logging.INFO)

    # 文件处理器（自动轮转）
    file_handler = RotatingFileHandler(
        'logs/firecrawl.log',
        maxBytes=10*1024*1024, # 10MB
        backupCount=5
    )
    file_handler.setLevel(logging.INFO)

    # 错误日志单独记录
    error_handler = RotatingFileHandler(
        'logs/firecrawl_errors.log',
        maxBytes=10*1024*1024,
        backupCount=5
    )
    error_handler.setLevel(logging.ERROR)

    # 格式化
    formatter = logging.Formatter(
        '%(asctime)s - %(name)s - %(levelname)s - %(message)s'
    )
    file_handler.setFormatter(formatter)
    error_handler.setFormatter(formatter)

    logger.addHandler(file_handler)
    logger.addHandler(error_handler)

    return logger

logger = setup_logging()

# 使用日志

```

```
def logged_scrape(url: str, **kwargs):  
    """带日志的爬取"""  
    try:  
        logger.info(f"开始爬取: {url}")  
        result = app.scrape(url, **kwargs)  
        logger.info(f"爬取成功: {url}")  
        return result  
  
    except Exception as e:  
        logger.error(  
            f"爬取失败: {url}",  
            extra={  
                'url': url,  
                'error': str(e),  
                'params': kwargs  
            },  
            exc_info=True  
        )  
        raise
```

性能优化

1. 连接池管理


```

from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

class OptimizedFirecrawlApp:
    """优化的 Firecrawl 客户端"""

    def __init__(self, api_key: str):
        self.api_key = api_key

        # 配置连接池
        self.session = self._create_session()
        self.app = FirecrawlApp(api_key=api_key)

    def _create_session(self):
        """创建优化的 Session"""
        session = requests.Session()

        # 重试策略
        retry_strategy = Retry(
            total=3,
            backoff_factor=1,
            status_forcelist=[429, 500, 502, 503, 504],
            allowed_methods=["GET", "POST"]
        )

        # HTTP 适配器
        adapter = HTTPAdapter(
            pool_connections=100,    # 连接池大小
            pool_maxsize=100,
            max_retries=retry_strategy,
            pool_block=False
        )

        session.mount("http://", adapter)
        session.mount("https://", adapter)

    return session

```

2. 响应压缩

```
# 启用 gzip 压缩
result = app.scrape(
    url="https://example.com",
    headers={
        'Accept-Encoding': 'gzip, deflate',
    }
)
```

3. 性能监控

```

import time
from contextlib import contextmanager

class PerformanceMonitor:
    """性能监控器"""

    def __init__(self):
        self.metrics = {
            'total_requests': 0,
            'total_time': 0.0,
            'avg_time': 0.0,
            'min_time': float('inf'),
            'max_time': 0.0,
        }

    @contextmanager
    def track(self, operation: str):
        """追踪操作性能"""
        start_time = time.time()

        try:
            yield
        finally:
            elapsed = time.time() - start_time
            self._record(operation, elapsed)

    def _record(self, operation: str, elapsed: float):
        """记录性能数据"""
        self.metrics['total_requests'] += 1
        self.metrics['total_time'] += elapsed
        self.metrics['avg_time'] = self.metrics['total_time'] / self.metrics['total_requests']
        self.metrics['min_time'] = min(self.metrics['min_time'], elapsed)
        self.metrics['max_time'] = max(self.metrics['max_time'], elapsed)

        logger.info(
            f"[{operation}] 耗时: {elapsed:.2f}s "
            f"(平均: {self.metrics['avg_time']:.2f}s)"
        )

    def get_report(self) -> Dict:

```

```
        """生成性能报告"""
        return {
            'total_requests': self.metrics['total_requests'],
            'total_time': round(self.metrics['total_time'], 2),
            'avg_time': round(self.metrics['avg_time'], 2),
            'min_time': round(self.metrics['min_time'], 2),
            'max_time': round(self.metrics['max_time'], 2),
            'requests_per_second': round(
                self.metrics['total_requests'] / self.metrics['total_time'], 2
            ) if self.metrics['total_time'] > 0 else 0
        }

# 使用性能监控
monitor = PerformanceMonitor()

def monitored_scrape(url: str):
    """带性能监控的爬取"""
    with monitor.track('scrape'):
        return app.scrape(url)
```

安全规范

1. 请求头安全

推荐的请求头配置

```
SECURE_HEADERS = {  
    'User-Agent': 'HawaiiHub/1.0 (Firecrawl Bot)',  
    'Accept': 'text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8',  
    'Accept-Language': 'en-US,en;q=0.9,zh-CN;q=0.8',  
    'Accept-Encoding': 'gzip, deflate',  
    'DNT': '1', # Do Not Track  
    'Connection': 'keep-alive',  
    'Upgrade-Insecure-Requests': '1',  
}  
  
result = app.scrape(  
    url="https://example.com",  
    headers=SECURE_HEADERS  
)
```

2. URL 验证

```

from urllib.parse import urlparse
import re

class URLValidator:
    """URL 验证器"""

    # 允许的协议
    ALLOWED_SCHEMES = ['http', 'https']

    # 禁止的域名（内网、敏感域名）
    BLOCKED_DOMAINS = [
        'localhost',
        '127.0.0.1',
        '0.0.0.0',
        '::1',
        '192.168.',
        '10.',
        '172.16.',
    ]

    # 允许的域名白名单（可选）
    ALLOWED_DOMAINS = [
        'hawaiinewsnow.com',
        'staradvertiser.com',
        'civilbeat.org',
        # ... 其他可信域名
    ]

    @classmethod
    def validate(cls, url: str, strict: bool = False) -> bool:
        """
        验证 URL

        参数:
            url: 待验证的 URL
            strict: 严格模式（只允许白名单域名）
        """
        try:
            parsed = urlparse(url)

```



```

# 检查协议
if parsed.scheme not in cls.ALLOWED_SCHEMES:
    raise ValueError(f"不允许的协议: {parsed.scheme}")

# 检查域名
hostname = parsed.hostname or ''

# 禁止列表检查
for blocked in cls.BLOCKED_DOMAINS:
    if hostname.startswith(blocked):
        raise ValueError(f"禁止访问的域名: {hostname}")

# 严格模式: 白名单检查
if strict and not any(
    hostname.endswith(domain)
    for domain in cls.ALLOWED_DOMAINS
):
    raise ValueError(f"域名不在白名单中: {hostname}")

return True

except Exception as e:
    logger.warning(f"URL 验证失败: {url} - {e}")
    return False

# 使用 URL 验证
def safe_scrape_validated(url: str, **kwargs):
    """验证 URL 后再爬取"""
    if not URLValidator.validate(url):
        raise ValueError(f"URL 验证失败: {url}")

    return app.scrape(url, **kwargs)

```

3. 数据脱敏

```

import re

class DataSanitizer:
    """数据脱敏器"""

    # 敏感信息正则
    PATTERNS = {
        'email': r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b',
        'phone': r'\b\d{3}[-.]?\d{3}[-.]?\d{4}\b',
        'ssn': r'\b\d{3}-\d{2}-\d{4}\b',
        'credit_card': r'\b\d{4}[-\s]?\d{4}[-\s]?\d{4}[-\s]?\d{4}\b',
    }

    @classmethod
    def sanitize(cls, text: str, keep_structure: bool = True) -> str:
        """
        脱敏文本中的敏感信息

        参数:
            text: 原始文本
            keep_structure: 是否保留结构 (如 abc@example.com -> ***@***.com)
        """
        for pattern_type, pattern in cls.PATTERNS.items():
            if keep_structure and pattern_type == 'email':
                # 保留邮箱结构
                text = re.sub(
                    pattern,
                    lambda m: cls._mask_email(m.group(0)),
                    text
                )
            else:
                # 完全替换
                text = re.sub(pattern, f'[{pattern_type.upper()}_REDACTED]', text)

        return text

    @staticmethod
    def _mask_email(email: str) -> str:
        """脱敏邮箱地址"""
        parts = email.split('@')

```

```
if len(parts) != 2:
    return '[EMAIL_REDACTED]'

username, domain = parts
masked_username = username[0] + '***' if username else '***'

domain_parts = domain.split('.')
masked_domain = '.'.join([
    '***' if i < len(domain_parts) - 1 else part
    for i, part in enumerate(domain_parts)
])

return f"{masked_username}@{masked_domain}"

# 使用数据脱敏
def scrape_and_sanitize(url: str):
    """爬取并脱敏"""
    result = app.scrape(url, formats=['markdown'])
    result['markdown'] = DataSanitizer.sanitize(result['markdown'])
    return result
```

监控与日志

1. 实时监控仪表板

```

from collections import defaultdict
from datetime import datetime, timedelta

class DashboardMetrics:
    """仪表板指标"""

    def __init__(self):
        self.metrics = defaultdict(lambda: {
            'count': 0,
            'success': 0,
            'failure': 0,
            'total_time': 0.0,
        })
        self.hourly_stats = defaultdict(int)

    def record(
        self,
        operation: str,
        success: bool,
        elapsed: float
    ):
        """记录操作指标"""
        m = self.metrics[operation]
        m['count'] += 1
        m['success' if success else 'failure'] += 1
        m['total_time'] += elapsed

        # 小时统计
        hour_key = datetime.now().strftime('%Y-%m-%d %H:00')
        self.hourly_stats[hour_key] += 1

    def get_dashboard(self) -> Dict:
        """获取仪表板数据"""
        dashboard = {}

        for operation, m in self.metrics.items():
            success_rate = (m['success'] / m['count'] * 100) if m['count'] > 0 else 0
            avg_time = m['total_time'] / m['count'] if m['count'] > 0 else 0

            dashboard[operation] = {

```

```

        '总请求': m['count'],
        '成功': m['success'],
        '失败': m['failure'],
        '成功率': f"{success_rate:.1f}%",
        '平均耗时': f"{avg_time:.2f}s",
    }

    return dashboard

def print_dashboard(self):
    """打印仪表板"""
    print("\n" + "=" * 80)
    print("📊 Firecrawl API 实时监控仪表板")
    print("=" * 80)

    dashboard = self.get_dashboard()
    for operation, stats in dashboard.items():
        print(f"\n{operation.upper()}:")
        for key, value in stats.items():
            print(f"    {key}: {value}")

    # 小时趋势
    print("\n每小时请求趋势:")
    for hour in sorted(self.hourly_stats.keys())[-24:]: # 最近 24 小时
        bar = '█' * (self.hourly_stats[hour] // 10)
        print(f"    {hour}: {bar} ({self.hourly_stats[hour]})")

    print("=" * 80)

# 全局仪表板
dashboard = DashboardMetrics()

# 定时打印 (每 5 分钟)
import threading

def print_dashboard_periodically():
    """定时打印仪表板"""
    while True:
        time.sleep(300) # 5 分钟
        dashboard.print_dashboard()

```

```
dashboard_thread = threading.Thread(target=print_dashboard_periodically, daemon=True)
dashboard_thread.start()
```


2. 告警系统

```

from enum import Enum

class AlertLevel(Enum):
    """告警级别"""
    INFO = "信息"
    WARNING = "警告"
    ERROR = "错误"
    CRITICAL = "严重"

class AlertSystem:
    """告警系统"""

    def __init__(
        self,
        webhook_url: Optional[str] = None, # Slack/钉钉 Webhook
        email_config: Optional[Dict] = None
    ):
        self.webhook_url = webhook_url
        self.email_config = email_config
        self.alert_history = []

    def alert(
        self,
        level: AlertLevel,
        title: str,
        message: str,
        context: Optional[Dict] = None
    ):
        """发送告警"""
        alert = {
            'level': level,
            'title': title,
            'message': message,
            'context': context or {},
            'timestamp': datetime.now()
        }

        self.alert_history.append(alert)

        # 根据级别决定是否发送通知

```

```

    if level in [AlertLevel.ERROR, AlertLevel.CRITICAL]:
        self._send_notification(alert)

# 记录日志
logger.log(
    logging.ERROR if level == AlertLevel.ERROR else logging.WARNING,
    f"[{level.value}] {title}: {message}",
    extra=context
)

def _send_notification(self, alert: Dict):
    """发送通知 (Webhook/邮件) """
    if self.webhook_url:
        try:
            import requests
            requests.post(self.webhook_url, json={
                'text': f"[{alert['level'].value}] {alert['title']}\n{alert['message']}"
            })
        except Exception as e:
            logger.error(f"发送告警失败: {e}")

# 使用告警系统
alert_system = AlertSystem(
    webhook_url=os.getenv('ALERT_WEBHOOK_URL')
)

# 示例: 成本告警
def check_cost_alert():
    """检查成本告警"""
    current_cost = cost_tracker.get_cost()

    if current_cost > 8.0: # 接近每日预算 $10
        alert_system.alert(
            AlertLevel.WARNING,
            "API 成本告警",
            f"今日成本已达 ${current_cost:.2f}, 接近预算上限",
            {'current_cost': current_cost, 'budget': 10.0}
        )

    if current_cost > 10.0: # 超过预算
        alert_system.alert(

```

```
AlertLevel.CRITICAL,  
"API 成本超支",  
f"今日成本 ${current_cost:.2f} 已超过预算 $10.00",  
{'current_cost': current_cost, 'budget': 10.0}  
)
```



代码规范

1. 统一的 API 调用封装

```
# utils/firecrawl_client.py
from typing import List, Dict, Optional, Union
from firecrawl import FirecrawlApp

class FirecrawlClient:
    """统一的 Firecrawl 客户端"""

    def __init__(self, api_key: Optional[str] = None):
        self.api_key = api_key or os.getenv('FIRECRAWL_API_KEY')
        self.app = FirecrawlApp(api_key=self.api_key)

        # 集成所有工具
        self.cache = SmartCache()
        self.cost_tracker = CostTracker()
        self.monitor = PerformanceMonitor()
        self.alert = AlertSystem()

    def scrape(
        self,
        url: str,
        formats: List[str] = ['markdown'],
        use_cache: bool = True,
        **kwargs
    ) -> Dict:
        """
        爬取单个 URL

        参数:

            url: 目标 URL
            formats: 返回格式
            use_cache: 是否使用缓存
            **kwargs: 其他参数

        返回:

            爬取结果字典
        """
        # URL 验证
        if not URLValidator.validate(url):
            raise ValueError(f"无效 URL: {url}")
```

```

# 尝试缓存
if use_cache:
    cached = self.cache.get(url, kwargs)
    if cached:
        return cached

# 执行爬取 (带监控)
with self.monitor.track('scrape'):
    try:
        result = self.app.scrape(
            url=url,
            formats=formats,
            **kwargs
        )

        # 记录成本
        self.cost_tracker.track('scrape', 1)

        # 保存缓存
        if use_cache:
            self.cache.set(url, kwargs, result)

        return result

    except Exception as e:
        # 错误处理
        error = ErrorHandler.handle_error(e, url)
        self.alert.alert(
            AlertLevel.ERROR,
            f"爬取失败: {url}",
            str(error)
        )
        raise error

def batch_scrape(
    self,
    urls: List[str],
    formats: List[str] = ['markdown'],
    batch_size: int = 10,
    **kwargs
) -> List[Dict]:

```

```
"""
```

批量爬取

参数:

```

    urls: URL 列表
    formats: 返回格式
    batch_size: 批次大小
    **kwargs: 其他参数

```

```
"""
```

```
# URL 去重
```

```

deduplicator = URLDeduplicator()
unique_urls = [
    url for url in urls
    if not deduplicator.is_duplicate_url(url)
]

```

```
# 分批处理
```

```

optimizer = FirecrawlOptimizer(self.api_key, batch_size)
results = optimizer.batch_scrape_optimized(
    unique_urls,
    formats=formats,
    **kwargs
)

```

```
# 记录成本
```

```
self.cost_tracker.track('scrape', len(results))
```

```
return results
```

```
def get_metrics(self) -> Dict:
```

```
    """获取所有指标"""
```

```

    return {
        'cost': self.cost_tracker.get_report(),
        'performance': self.monitor.get_report(),
        'cache_stats': {
            # TODO: 实现缓存统计
        }
    }

```

```
# 全局客户端实例
```

```
firecrawl_client = FirecrawlClient()
```


使用示例

```
def scrape_hawaii_news():
```

```
    """爬取夏威夷新闻"""
```

```
    urls = [
```

```
        "https://www.hawaiinewsnow.com/",
```

```
        "https://www.staradvertiser.com/",
```

```
    ]
```

```
    results = firecrawl_client.batch_scrape(urls)
```

打印指标

```
metrics = firecrawl_client.get_metrics()
```

```
print(f"成本: ${metrics['cost']['total_cost']}")
```

```
print(f"性能: {metrics['performance']['avg_time']:.2f}s")
```

```
return results
```

2. 配置文件规范

```

# config/firecrawl_config.py
from dataclasses import dataclass
from typing import List, Optional

@dataclass
class FirecrawlConfig:
    """Firecrawl 配置"""

    # API 配置
    api_key: str
    api_url: str = "https://api.firecrawl.dev"
    timeout: int = 60
    max_retries: int = 3

    # 功能开关
    enable_cache: bool = True
    enable_monitoring: bool = True
    enable_cost_tracking: bool = True

    # 缓存配置
    cache_ttl: int = 3600
    cache_dir: str = "./cache"

    # 成本限制
    daily_budget: float = 10.0
    monthly_budget: float = 200.0

    # 性能配置
    batch_size: int = 10
    max_workers: int = 5
    rate_limit: int = 10

    # 告警配置
    alert_webhook: Optional[str] = None
    alert_email: Optional[str] = None

    @classmethod
    def from_env(cls) -> 'FirecrawlConfig':
        """从环境变量加载配置"""
        return cls(

```

```
api_key=os.getenv('FIRECRAWL_API_KEY'),
api_url=os.getenv('FIRECRAWL_API_URL', cls.api_url),
timeout=int(os.getenv('FIRECRAWL_TIMEOUT', cls.timeout)),
enable_cache=os.getenv('FIRECRAWL_ENABLE_CACHE', 'true').lower() == 'true',
daily_budget=float(os.getenv('FIRECRAWL_DAILY_BUDGET', cls.daily_budget)),
alert_webhook=os.getenv('FIRECRAWL_ALERT_WEBHOOK'),
)

def validate(self):
    """验证配置"""
    if not self.api_key:
        raise ValueError("未设置 FIRECRAWL_API_KEY")

    if self.daily_budget <= 0:
        raise ValueError("每日预算必须大于 0")

    if self.batch_size < 1:
        raise ValueError("批次大小必须至少为 1")

    return True

# 加载配置
config = FirecrawlConfig.from_env()
config.validate()
```

应急预案

1. API 故障应急

```

class EmergencyFallback:
    """应急后备方案"""

    def __init__(self):
        # 备用 API 密钥
        self.backup_keys = [
            os.getenv('FIRECRAWL_API_KEY_BACKUP_1'),
            os.getenv('FIRECRAWL_API_KEY_BACKUP_2'),
        ]
        self.current_key_index = 0

        # 降级策略
        self.degraded_mode = False

    def switch_to_backup_key(self):
        """切换到备用密钥"""
        if self.current_key_index < len(self.backup_keys) - 1:
            self.current_key_index += 1
            new_key = self.backup_keys[self.current_key_index]
            print(f"⚠️ 切换到备用密钥 #{self.current_key_index + 1}")
            return FirecrawlApp(api_key=new_key)

        print("❌ 所有备用密钥已耗尽, 启动降级模式")
        self.degraded_mode = True
        return None

    def scrape_with_fallback(self, url: str, **kwargs):
        """带降级的爬取"""
        if self.degraded_mode:
            # 降级模式: 使用简单的 requests
            return self._simple_scrape(url)

        try:
            return app.scrape(url, **kwargs)

        except Exception as e:
            # API 故障, 尝试切换密钥
            backup_app = self.switch_to_backup_key()

            if backup_app:

```

```

        return backup_app.scrape(url, **kwargs)
    else:
        # 降级到简单爬取
        return self._simple_scrape(url)

    @staticmethod
    def _simple_scrape(url: str) -> Dict:
        """简单爬取（降级方案）"""
        import requests
        from bs4 import BeautifulSoup

        response = requests.get(url, timeout=30)
        soup = BeautifulSoup(response.text, 'html.parser')

        # 提取主要内容
        main_content = soup.find('main') or soup.find('article') or soup.body
        text = main_content.get_text(strip=True) if main_content else ""

        return {
            'url': url,
            'markdown': text,
            'method': 'fallback',
            'warning': '使用降级方案，内容可能不完整'
        }

emergency = EmergencyFallback()

```

2. 速率限制应对

```

class RateLimiter:
    """速率限制器"""

    def __init__(self, max_requests: int = 100, time_window: int = 60):
        self.max_requests = max_requests
        self.time_window = time_window
        self.requests = []

    def wait_if_needed(self):
        """如果需要，等待直到可以发送请求"""
        now = time.time()

        # 清理过期请求
        self.requests = [
            req_time for req_time in self.requests
            if now - req_time < self.time_window
        ]

        # 检查是否超限
        if len(self.requests) >= self.max_requests:
            # 计算需要等待的时间
            oldest_request = min(self.requests)
            wait_time = self.time_window - (now - oldest_request)

            if wait_time > 0:
                print(f"⌚ 达到速率限制，等待 {wait_time:.1f} 秒...")
                time.sleep(wait_time)

        # 记录本次请求
        self.requests.append(time.time())

rate_limiter = RateLimiter(max_requests=100, time_window=60)

def rate_limited_scrape(url: str, **kwargs):
    """速率限制的爬取"""
    rate_limiter.wait_if_needed()
    return app.scrape(url, **kwargs)

```


快速参考

常用命令

```
# 安装依赖
pip install firecrawl-py requests python-dotenv

# 运行测试
python -m pytest tests/

# 查看成本报告
python -m scripts.cost_report

# 清理缓存
python -m scripts.clear_cache

# 导出日志
python -m scripts.export_logs --date 2025-10-27
```

环境变量清单

```
# 必需
FIRECRAWL_API_KEY=fc-xxx

# 可选
FIRECRAWL_API_URL=https://api.firecrawl.dev
FIRECRAWL_TIMEOUT=60
FIRECRAWL_MAX_RETRIES=3
FIRECRAWL_ENABLE_CACHE=true
FIRECRAWL_CACHE_TTL=3600
FIRECRAWL_DAILY_BUDGET=10.0
FIRECRAWL_MONTHLY_BUDGET=200.0
FIRECRAWL_ALERT_WEBHOOK=https://hooks.slack.com/xxx
```

性能基准

操作	单次耗时	批量 (10个)	成本
Scrape	1-3s	10-15s	\$0.005/页
Search	2-5s	-	\$0.01/次
Crawl	30-60s	-	\$0.004/页

检查清单

上线前检查

- ☐ API 密钥已配置且有效
- ☐ 环境变量完整
- ☐ 错误处理已实现
- ☐ 日志系统已配置
- ☐ 成本监控已启用
- ☐ 缓存策略已配置
- ☐ 速率限制已设置
- ☐ 告警系统已测试
- ☐ 备用方案已准备
- ☐ 文档已更新

日常检查

- ☐ 检查 API 配额使用情况
- ☐ 查看错误日志
- ☐ 检查成本是否超标
- ☐ 清理过期缓存
- ☐ 更新监控数据
- ☐ 备份重要数据

相关文档

- [Firecrawl 官方文档](#)
 - [Firecrawl 生态系统指南](#)
 - [HawaiiHub 技术规范](#)
-

🔥 遵守这些规范，让 **Firecrawl 云 API** 为 **HawaiiHub** 提供强大且稳定的数据采集能力！🌴

最后更新: 2025-10-27

维护者: *HawaiiHub AI Team*

版本: *v1.0.0*



Firecrawl 云 API 快速配置指南

专为 HawaiiHub 团队打造

配置时间: 10 分钟

难度: ★★☆☆☆



核心文档

文档	用途	适合人群
FIRECRAWL_CLOUD_API_RULES.md	完整使用规范 (本规范)	所有开发者 ★★★★★
FIRECRAWL_README.md	项目入口	新手
FIRECRAWL_QUICK_INDEX.md	快速索引	查找项目
FIRECRAWL_ECOSYSTEM_GUIDE.md	生态系统	深入学习



5 分钟快速配置

步骤 1: 环境变量配置

创建 `.env` 文件:

```
# 复制模板
cp .env.example .env

# 编辑配置
vim .env
```

添加以下内容:

```
# =====
# Firecrawl 云API 配置
# =====

# 🔑 必需: API 密钥
FIRECRAWL_API_KEY=fc-your-api-key-here

# 🌐 API 端点 (通常不需要修改)
FIRECRAWL_API_URL=https://api.firecrawl.dev

# ⌚ 超时设置 (秒)
FIRECRAWL_TIMEOUT=60

# 🔄 重试次数
FIRECRAWL_MAX_RETRIES=3

# 📁 缓存配置
FIRECRAWL_ENABLE_CACHE=true
FIRECRAWL_CACHE_TTL=3600

# 💰 成本限制
FIRECRAWL_DAILY_BUDGET=10.0
FIRECRAWL_MONTHLY_BUDGET=200.0

# 📢 告警 (可选)
FIRECRAWL_ALERT_WEBHOOK=https://hooks.slack.com/your-webhook

# 🗑️ 备用密钥 (可选)
FIRECRAWL_API_KEY_BACKUP_1=fc-backup-key-1
FIRECRAWL_API_KEY_BACKUP_2=fc-backup-key-2
```

步骤 2: 安装依赖

```
pip install firecrawl-py python-dotenv requests
```

步骤 3: 测试连接

创建 `test_firecrawl.py`:

```
import os
from dotenv import load_dotenv
from firecrawl import FirecrawlApp

# 加载环境变量
load_dotenv()

# 初始化客户端
app = FirecrawlApp(api_key=os.getenv('FIRECRAWL_API_KEY'))

# 测试爬取
result = app.scrape(
    url="https://www.hawaiinewsnow.com/",
    formats=['markdown'],
    onlyMainContent=True
)

print("✅ 连接成功! ")
print(f"标题: {result.get('title', 'N/A')}")
print(f"内容长度: {len(result.get('markdown', ''))} 字符")
```

运行测试:

```
python test_firecrawl.py
```

核心功能速览

1. 基础爬取

```
from firecrawl import FirecrawlApp
import os

app = FirecrawlApp(api_key=os.getenv('FIRECRAWL_API_KEY'))

# 单页爬取
result = app.scrape(
    url="https://example.com",
    formats=['markdown'],
    onlyMainContent=True,      # ✅ 只要主要内容
    maxAge=3600000            # ✅ 使用 1 小时缓存
)

# 批量爬取
results = app.batch_scrape(
    urls=["https://example.com/1", "https://example.com/2"],
    formats=['markdown']
)
```

2. 搜索功能

```
# 搜索夏威夷新闻
results = app.search(
    query="Hawaii news 夏威夷 华人社区",
    sources=[{"type": "web"}],
    limit=10
)
```

3. 变更监控

```
# 监控网站变化
result = app.scrape(
    url="https://example.com",
    formats=['markdown', {'type': 'changeTracking'}]
)
```

最佳实践（必读）

DO - 推荐做法

1. 使用批量接口

```
#  好 - 批量处理
results = app.batch_scrape(urls)

#  差 - 循环调用
for url in urls:
    result = app.scrape(url)
```

2. 启用缓存

```
#  好 - 使用缓存
result = app.scrape(url, maxAge=3600000)

#  差 - 不使用缓存
result = app.scrape(url)
```


3. 过滤内容

```
# ✅好 - 只要主要内容
result = app.scrape(
    url,
    onlyMainContent=True,
    removeTags=['script', 'style']
)

# ❌差 - 获取所有内容（浪费）
result = app.scrape(url)
```

4. 错误处理

```
# ✅好 - 完整的错误处理
try:
    result = app.scrape(url)
except Exception as e:
    logger.error(f"爬取失败: {url} - {e}")
    # 降级处理
    result = fallback_scrape(url)

# ❌差 - 不处理错误
result = app.scrape(url)
```

5. 成本监控

```
# ✅好 - 追踪成本
@track_cost('scrape')
def scrape_urls(urls):
    return app.batch_scrape(urls)

# 定期检查
print(f"今日成本: ${cost_tracker.get_cost()}")
```

❌ DON'T - 禁止行为

1. 硬编码密钥

```
# ❌ 严禁
app = FirecrawlApp(api_key="fc-xxxxxxx")

# ✅ 正确
app = FirecrawlApp(api_key=os.getenv('FIRECRAWL_API_KEY'))
```

2. 不验证 URL

```
# ❌ 危险
result = app.scrape(user_input_url)

# ✅ 安全
if URLValidator.validate(user_input_url):
    result = app.scrape(user_input_url)
```

3. 忽略速率限制

```
# ❌ 会被限流
for url in urls:
    app.scrape(url)

# ✅ 速率控制
rate_limiter.wait_if_needed()
app.scrape(url)
```

4. 不控制成本

```
# ❌ 可能超支
app.crawl(url, limit=1000)

# ✅ 控制预算
if budget.check_budget(estimated_cost):
    app.crawl(url, limit=100)
```

成本预算参考

定价（实际以官方为准）

操作	单价	说明
Scrape	~\$0.005/页	单页爬取
Crawl	~\$0.004/页	批量爬取（有折扣）
Search	~\$0.01/次	搜索功能
Extract	~\$0.008/次	结构化提取

HawaiiHub 预算建议

阶段	每日预算	每月预算	预估用量
MVP	\$10	\$200	2K 页/天
增长期	\$20	\$500	5K 页/天
成熟期	\$50	\$1000	12K 页/天

省钱技巧

1. **充分利用缓存**: 相同内容 1 小时内不重复爬取
2. **批量处理**: 使用 `batch_scrape` 获得折扣
3. **过滤内容**: 只爬取需要的部分
4. **定时任务**: 错峰执行 (夜间)
5. **去重**: 避免重复爬取相同 URL



关键配置

1. 成本告警

```
# config/alerts.py
from firecrawl_utils import cost_tracker, alert_system

def check_daily_budget():
    """每小时检查一次预算"""
    cost = cost_tracker.get_cost()

    if cost > 8.0: # 80% 预算
        alert_system.alert(
            level=AlertLevel.WARNING,
            title="成本告警",
            message=f"今日成本 ${cost:.2f}, 接近预算上限"
        )
```

2. 自动重试

```
# config/retry.py
from firecrawl_utils import retry_with_exponential_backoff

@retry_with_exponential_backoff(max_retries=5)
def scrape_with_retry(url):
    return app.scrape(url)
```

3. 速率限制

```
# config/rate_limit.py
from firecrawl_utils import RateLimiter

rate_limiter = RateLimiter(
    max_requests=100, # 每分钟 100 次
    time_window=60
)

def rate_limited_scrape(url):
    rate_limiter.wait_if_needed()
    return app.scrape(url)
```



监控仪表板

实时指标

```
# 打印实时指标
from firecrawl_utils import dashboard

dashboard.print_dashboard()
```

输出示例：

 Firecrawl API 实时监控仪表板

SCRAPE:

总请求: 150
成功: 145
失败: 5
成功率: 96.7%
平均耗时: 1.85s

SEARCH:

总请求: 20
成功: 20
失败: 0
成功率: 100.0%
平均耗时: 3.12s

每小时请求趋势:

2025-10-27 08:00:  (80)
2025-10-27 09:00:  (60)
2025-10-27 10:00:  (100)

常用代码片段

新闻采集

```
# scripts/scrape_news.py
from firecrawl_utils import firecrawl_client

def scrape_hawaii_news():
    """爬取夏威夷新闻"""
    news_sites = [
        "https://www.hawaiinewsnow.com/",
        "https://www.staradvertiser.com/",
        "https://www.civilbeat.org/"
    ]

    results = firecrawl_client.batch_scrape(
        urls=news_sites,
        formats=['markdown'],
        onlyMainContent=True
    )

    # 保存结果
    for result in results:
        save_to_database(result)

    # 打印指标
    metrics = firecrawl_client.get_metrics()
    print(f"✅ 爬取完成! 成本: ${metrics['cost']['total_cost']}")

    return results

if __name__ == "__main__":
    scrape_hawaii_news()
```

定时任务

```
# crontab 配置
# 每小时爬取一次新闻
0 * * * * cd /path/to/project && python scripts/scrape_news.py

# 每日成本报告（早上 9 点）
0 9 * * * cd /path/to/project && python scripts/daily_report.py

# 每周清理缓存（周日凌晨 2 点）
0 2 * * 0 cd /path/to/project && python scripts/clear_cache.py
```

完整代码示例

HawaiiHub 新闻爬虫

```

# hawaiihub/news_scraper.py
import os
from typing import List, Dict
from dotenv import load_dotenv
from firecrawl import FirecrawlApp

# 加载环境变量
load_dotenv()

class HawaiiNewsScraper:
    """夏威夷新闻爬虫"""

    # 新闻源配置
    NEWS_SOURCES = {
        'hawaii_news_now': 'https://www.hawaiinewsnow.com/',
        'star_advertiser': 'https://www.staradvertiser.com/',
        'civil_beat': 'https://www.civilbeat.org/',
    }

    def __init__(self):
        self.app = FirecrawlApp(api_key=os.getenv('FIRECRAWL_API_KEY'))

    def scrape_all_news(self) -> List[Dict]:
        """爬取所有新闻源"""
        results = []

        for source_name, url in self.NEWS_SOURCES.items():
            try:
                print(f"🔄 爬取 {source_name}...")

                # 使用 Search 发现新闻
                search_results = self.app.search(
                    query=f"site:{url} Hawaii news",
                    limit=10
                )

                # 批量爬取完整内容
                urls = [r['url'] for r in search_results]
                articles = self.app.batch_scrape(
                    urls=urls,

```

```

        formats=['markdown'],
        onlyMainContent=True
    )

    # 添加来源标记
    for article in articles:
        article['source'] = source_name

    results.extend(articles)
    print(f"✅ {source_name}: {len(articles)} 篇文章")

except Exception as e:
    print(f"❌ {source_name} 失败: {e}")

return results

def save_to_database(self, articles: List[Dict]):
    """保存到数据库"""
    # TODO: 实现数据库保存逻辑
    pass

# 使用示例
if __name__ == "__main__":
    scraper = HawaiiNewsScraper()
    articles = scraper.scrape_all_news()

    print(f"\n📊 总计爬取 {len(articles)} 篇文章")

    # 保存到数据库
    scraper.save_to_database(articles)

```

故障排查

常见问题

Q1: **Authentication failed** 错误

原因: API 密钥无效或过期

解决:

```
# 检查环境变量
echo $FIRECRAWL_API_KEY

# 重新设置
export FIRECRAWL_API_KEY=fc-your-new-key

# 验证
python test_firecrawl.py
```

Q2: **Rate limit exceeded** 错误

原因: 超过速率限制

解决:

```
# 添加速率限制
from firecrawl_utils import rate_limiter

rate_limiter.wait_if_needed()
result = app.scrape(url)
```

Q3: **Request timeout** 错误

原因: 请求超时

解决:

```
# 增加超时时间
```

```
export FIRECRAWL_TIMEOUT=120
```

Q4: 成本过高

原因: 未优化请求

解决:

1. 启用缓存
2. 使用批量接口
3. 过滤不需要的内容
4. 去重 URL

上线检查清单

配置检查

- ☐  `.env` 文件已配置
- ☐  API 密钥已验证
- ☐  备用密钥已设置
- ☐  预算限制已配置
- ☐  告警 Webhook 已测试

功能检查

- ☐  基础爬取功能正常
- ☐  批量爬取功能正常
- ☐  缓存功能启用
- ☐  错误处理完整
- ☐  日志记录正常

监控检查

- ☐  成本监控已启用

- ☐ ☒ 性能监控已启用
- ☐ ☒ 告警系统已测试
- ☐ ☒ 仪表板可访问

安全检查

- ☐ ☒ API 密钥安全存储
- ☐ ☒ URL 验证已实现
- ☐ ☒ 数据脱敏已配置
- ☐ ☒ `.env` 已加入 `.gitignore`

获取帮助

官方资源

-  [官方文档](#)
-  [Discord 社区](#)
-  [技术支持](#)

团队资源

-  完整规范: `FIRECRAWL_CLOUD_API_RULES.md`
-  快速索引: `FIRECRAWL_QUICK_INDEX.md`
-  生态系统: `FIRECRAWL_ECOSYSTEM_GUIDE.md`
-  团队规范: `/Users/zhiledeng/Documents/augment-projects/AGENTS.md`

下一步

1. **立即:** 完成环境配置 (10 分钟)
2. **今天:** 运行第一个爬虫测试
3. **本周:** 部署新闻采集系统

4. 本月: 优化成本和性能

🔥 开始使用 **Firecrawl** 云 **API**, 为 **HawaiiHub** 提供强大的数据采集能力! 🌴

创建时间: 2025-10-27

维护者: *HawaiiHub AI Team*

版本: *v1.0.0*

HawaiiHub 实战案例手册

创建时间: 2025-10-27

项目: HawaiiHub - 夏威夷华人社区平台

目标: 使用 Firecrawl 构建完整的数据采集和分析系统

难度: 初级 → 高级 (循序渐进)



手册简介

本手册以 **HawaiiHub** 项目为例，通过 **15 个实战案例**展示如何使用 Firecrawl 构建生产级数据采集系统。



学习目标

通过本手册，你将学会：

- ☒ 爬取夏威夷本地新闻网站
- ☒ 采集华人商家信息（餐厅、超市、服务）
- ☒ 监控租房价格变化
- ☒ 分析用户评论和反馈
- ☒ 提取招聘信息
- ☒ 构建 AI 驱动的智能推荐系统

案例统计

类别	案例数	技术栈
数据采集	5	Scrape、Crawl、Map
数据分析	3	Extract、Pandas、Charts
实时监控	2	Change Tracking、定时任务
AI 应用	3	LLM、RAG、推荐系统
完整系统	2	端到端解决方案
总计	15	Python + Next.js

案例组织

HawaiiHub实战案例手册.md	← 本文档（总览）
├─ 案例 01-05：数据采集基础	← 爬取各类数据
├─ 案例 06-08：数据分析	← 提取和分析
├─ 案例 09-10：实时监控	← 变更监控
├─ 案例 11-13：AI 应用	← 智能推荐
└─ 案例 14-15：完整系统	← 生产级系统

快速开始

前置条件

```
# 1. Python 环境
Python 3.11+

# 2. 安装依赖
pip3 install firecrawl-py python-dotenv requests pandas pydantic

# 3. 配置 API 密钥
cp env.template .env
# 编辑 .env, 添加 FIRECRAWL_API_KEY=fc-xxx

# 4. 验证环境
python3 -c "from firecrawl import FirecrawlApp; print('✅ 环境就绪')"
```

目录结构

```
hawaiihub-firecrawl-cases/
├─ case-01-news-scraper/      # 案例 01
├─ case-02-restaurant-collector/ # 案例 02
├─ case-03-housing-monitor/    # 案例 03
├─ ...
├─ data/                      # 数据存储
├─ utils/                     # 工具函数
└─ README.md
```

案例目录

第一部分：数据采集基础（案例 01-05）

案例 01: 夏威夷新闻采集器

业务需求: 自动采集夏威夷本地新闻，为社区提供最新资讯

涉及网站:

- Hawaii News Now: <https://www.hawaiinewsnow.com/>
- Honolulu Star-Advertiser: <https://www.staradvertiser.com/>
- Civil Beat: <https://www.civilbeat.org/>

核心功能:

- 使用 Scrape API 采集单篇新闻
- 使用 Crawl API 爬取完整新闻列表
- 提取标题、作者、时间、内容
- 保存为 JSON 和 Markdown

技术栈: Python + Firecrawl Scrape/Crawl API

难度:  (初级)

学习时间: 2 小时

参考项目:

- [blog-articles](#) (examples/blog-articles)
- [hacker_news_scraper](#) (examples/hacker_news_scraper)

案例 02: 华人商家信息采集

业务需求: 建立完整的夏威夷华人商家数据库

目标商家类型:

- 中餐厅
- 华人超市
- 中文学校
- 移民服务
- 房地产中介

采集来源:

- Yelp: https://www.yelp.com/search?find_desc=Chinese&find_loc=Honolulu
- Google Maps
- 本地分类信息网站

核心功能:

- 使用 Extract API 结构化提取
- 提取: 名称、地址、电话、营业时间、评分
- 验证数据完整性
- 去重和数据清洗

技术栈: Python + Firecrawl Extract API + Pydantic

难度: ★★★★★ (中级)

学习时间: 4 小时

参考项目:

- `company-data-scraper` (firecrawl-app-examples)
 - `crm_lead_enrichment` (examples/crm_lead_enrichment)
 - `gpt-4.1-company-researcher` (examples/gpt-4.1-company-researcher)
-

案例 03: 租房信息监控系统 ★★★★★

业务需求: 实时监控夏威夷租房市场, 提供价格预警

采集来源:

- Craigslist Honolulu: <https://honolulu.craigslist.org/search/apa>
- Zillow
- Apartments.com

核心功能:

- 使用 Crawl API 爬取房源列表
- 提取价格、面积、位置、设施
- 使用 Change Tracking 监控价格变化
- 价格低于预期时发送通知

技术栈: Python + Firecrawl Crawl + Change Tracking + Email

难度: ★★★★★ (中级)

学习时间: 4 小时

参考项目:

- `automated_price_tracking` (firecrawl-app-examples)

- `deep-research-apartment-finder` (examples/deep-research-apartment-finder)
 - `o3-mini-deal-finder` (examples/o3-mini-deal-finder)
-

案例 04: 招聘信息聚合 ★★★★★

业务需求: 为华人社区提供本地招聘信息

采集来源:

- Indeed Honolulu
- LinkedIn Jobs
- 本地华人论坛招聘板块

核心功能:

- 使用 Search API 搜索相关职位
- 提取职位名称、公司、薪资、要求
- 按类别分类（餐饮、零售、技术等）
- 生成每日招聘简报

技术栈: Python + Firecrawl Search + Extract API

难度: ★★★★★ (初级)

学习时间: 3 小时

参考项目:

- `ai-resume-job-matching` (firecrawl-app-examples)
 - `job-resource-analyzer` (examples/job-resource-analyzer)
 - `o1_job_recommender` (examples/o1_job_recommender)
-

案例 05: 活动信息爬取 ★★★★★

业务需求: 汇总夏威夷本地活动，方便华人参与

采集来源:

- Eventbrite Honolulu
- Facebook Events
- 本地社区网站

核心功能:

- 使用 Map API 发现所有活动页面
- 批量爬取活动详情

- 提取时间、地点、价格、主办方
- 按类型分类（文化、运动、音乐等）

技术栈: Python + Firecrawl Map + Batch Scrape

难度: ★★☆☆ (初级)

学习时间: 2 小时

参考项目:

- `blog-articles` (examples/blog-articles)
-

第二部分：数据分析（案例 06-08）

案例 06: 商家评论情感分析 ★★★★★

业务需求: 分析华人商家的用户评论，提取关键问题

分析维度:

- 情感分析（正面、负面、中立）
- 关键词提取（服务、价格、环境等）
- 评分趋势分析
- 改进建议生成

核心功能:

- 爬取 Yelp 评论
- 使用 LLM 进行情感分析
- 生成可视化报告
- 提供改进建议

技术栈: Python + Firecrawl + Claude/GPT + Pandas

难度: ★★★★★ (中级)

学习时间: 5 小时

参考项目:

- `review-analyzer` (firecrawl-app-examples)
 - `claude-3.7-stock-analyzer` (examples/claude-3.7-stock-analyzer)
-

案例 07: 租房市场数据分析 ★★★★★

业务需求: 分析租房市场趋势, 提供定价建议

分析内容:

- 不同区域价格对比
- 房型价格分布
- 价格随时间变化趋势
- 性价比分析

核心功能:

- 爬取历史租房数据
- 数据清洗和标准化
- 统计分析和可视化
- 生成市场报告

技术栈: Python + Firecrawl + Pandas + Matplotlib

难度: ★★★★★ (中级)

学习时间: 4 小时

参考项目:

- `scrape_and_analyze_airbnb_data_e2b` (examples/scrape_and_analyze_airbnb_data_e2b)
 - `visualize_website_topics_e2b` (examples/visualize_website_topics_e2b)
-

案例 08: 竞品分析系统 ★★★★★

业务需求: 监控竞品动态, 生成分析报告

分析对象:

- 其他华人社区平台
- 本地分类信息网站
- 社交媒体群组

核心功能:

- 定期爬取竞品网站
- 对比功能和内容
- 分析用户反馈
- 生成竞品分析报告

技术栈: Python + Firecrawl + LLM

难度: ★★★★★ (中级)

学习时间: 5 小时

参考项目:

- [search-competitor-analysis](#) (firecrawl-app-examples)
 - [gemini-github-analyzer](#) (examples/gemini-github-analyzer)
-

第三部分：实时监控（案例 09-10）

案例 09: 价格变动监控 ★★★★★

业务需求: 实时监控关键商品和服务价格

监控对象:

- 租房价格
- 二手车价格
- 商家促销信息

核心功能:

- 使用 Change Tracking 监控变化
- 设置价格阈值
- 自动发送通知（Email/SMS/微信）
- 保存历史价格数据

技术栈: Python + Firecrawl Change Tracking + 通知服务

难度: ★★★★★（中级）

学习时间: 3 小时

参考项目:

- [automated_price_tracking](#) (firecrawl-app-examples)
 - [change-detection-tutorial](#) (firecrawl-app-examples)
-

案例 10: 内容更新监控 ★★★★★

业务需求: 监控重要网站内容更新

监控对象:

- 政府公告
- 签证政策

- 本地新闻
- 社区活动

核心功能:

- 定时检查网站变化
- 提取新增内容
- 生成更新摘要
- 推送到社区

技术栈: Python + Firecrawl + 定时任务

难度: ★★★ (初级)

学习时间: 2 小时

参考项目:

- [change-detection-tutorial](#) (firecrawl-app-examples)
-

第四部分：AI 应用（案例 11-13）

案例 11: 智能问答系统 ★★★★★

业务需求: 构建 HawaiiHub 知识库问答系统

知识来源:

- 网站所有文章
- 用户常见问题
- 本地服务指南

核心功能:

- 爬取所有内容构建知识库
- 使用 RAG 技术实现智能问答
- 支持中英文问答
- 持续更新知识库

技术栈: Python + Firecrawl + LangChain + Vector DB

难度: ★★★★★ (高级)

学习时间: 8 小时

参考项目:

- [local-website-chatbot](#) (firecrawl-app-examples)
- [deepseek-rag](#) (firecrawl-app-examples)

- `web_data_rag_with_llama3` (examples/web_data_rag_with_llama3)
 - `website_qa_with_gemini_caching` (examples/website_qa_with_gemini_caching)
-

案例 12: 个性化推荐系统 ★★★★★

业务需求: 为用户推荐合适的商家和服务

推荐内容:

- 餐厅推荐
- 租房推荐
- 活动推荐
- 服务推荐

核心功能:

- 爬取用户浏览历史
- 分析用户偏好
- 使用协同过滤推荐
- 实时更新推荐结果

技术栈: Python + Firecrawl + ML 推荐算法

难度: ★★★★★ (高级)

学习时间: 10 小时

参考项目:

- `ai-resume-job-matching` (firecrawl-app-examples)
 - `o1_job_recommender` (examples/o1_job_recommender)
-

案例 13: AI 内容生成 ★★★★★

业务需求: 自动生成社区简报和内容

生成内容:

- 每周新闻简报
- 商家推荐文章
- 活动预告
- 生活指南

核心功能:

- 爬取相关内容

- 使用 LLM 生成文章
- 自动配图
- 定时发布

技术栈: Python + Firecrawl + GPT/Claude + CMS

难度: ★★★★★ (中级)

学习时间: 6 小时

参考项目:

- `aginews-ai-newsletter` (examples/aginews-ai-newsletter)
 - `deepseek-v3-agi-newsletter` (firecrawl-app-examples)
 - `ai-podcast-generator` (examples/ai-podcast-generator)
-

第五部分：完整系统（案例 14-15）

案例 14: 端到端商家管理系统 ★★★★★

业务需求: 完整的商家信息管理系统

系统功能:

1. 数据采集

- 自动爬取商家信息
- 更新营业时间、菜单、价格

1. 数据管理

- 商家信息存储
- 评论管理
- 图片管理

2. 数据展示

- 商家列表页面
- 详情页面
- 搜索和筛选

3. 监控告警

- 价格变动提醒
- 新评论通知
- 数据质量检查

技术栈:

- 后端: Python + Firecrawl + FastAPI + PostgreSQL

- 前端: Next.js + React + Tailwind CSS
- 部署: Docker + Vercel

难度: ★★★★★ (高级)

学习时间: 2-3 天

参考项目:

- `company-data-scraper` (firecrawl-app-examples)
 - `full_example_apps` (examples/full_example_apps)
-

案例 15: HawaiiHub 完整数据平台 ★★★★★

业务需求: HawaiiHub 生产级数据采集和分析平台

平台架构:



核心模块:

1. 数据采集模块

- 新闻采集
- 商家采集
- 租房采集
- 招聘采集
- 活动采集

2. 数据处理模块

- 数据清洗
- 去重验证

- 格式标准化
- AI 内容分析

3. 应用服务模块

- RESTful API
- GraphQL API
- WebSocket 实时推送
- 搜索服务

4. 智能功能模块

- 智能推荐
- 智能问答
- 内容生成
- 趋势分析

5. 运营管理模块

- 数据质量监控
- 爬取任务管理
- 用户行为分析
- 成本控制

技术栈:

- 后端: Python + FastAPI + Celery + Redis
- 前端: Next.js 14 + TypeScript + Tailwind CSS
- 数据库: PostgreSQL + Redis + Elasticsearch
- 爬虫: Firecrawl Cloud API
- **AI**: LangChain + OpenAI/Claude
- 部署: Docker + Kubernetes + Vercel
- 监控: Sentry + Grafana + Prometheus

难度: ★★★★★ (专家级)

学习时间: 1-2 周

参考项目:

- 所有前面的案例
- `full_example_apps` (examples/full_example_apps)
- `kubernetes` (examples/kubernetes)

案例详解

案例 01 详解：夏威夷新闻采集器

步骤 1: 分析目标网站

```
# 1.1 分析网站结构
# Hawaii News Now: https://www.hawaiinewsnow.com/

# 首页结构:
# - 头条新闻: class="lead-story"
# - 新闻列表: class="article-list"
# - 每篇新闻: <article>标签

# 新闻详情页结构:
# - 标题: <h1 class="headline">
# - 作者: <div class="byline">
# - 时间: <time datetime="">
# - 内容: <div class="article-body">
```

步骤 2: 爬取单篇新闻


```

from firecrawl import FirecrawlApp
import os
from datetime import datetime
import json

# 初始化客户端
app = FirecrawlApp(api_key=os.getenv("FIRECRAWL_API_KEY"))

def scrape_single_news(url: str) -> dict:
    """
    爬取单篇新闻

    Args:
        url: 新闻 URL

    Returns:
        新闻数据字典
    """
    try:
        # 使用 Scrape API
        result = app.scrape(
            url=url,
            formats=["markdown", "html"],
            only_main_content=True, # 只提取主要内容
            remove_base64_images=True, # 移除 base64 图片
        )

        # 提取数据
        news_data = {
            "url": url,
            "title": extract_title(result.markdown),
            "author": extract_author(result.markdown),
            "published_date": extract_date(result.markdown),
            "content": result.markdown,
            "html": result.html,
            "scraped_at": datetime.now().isoformat(),
        }

    return news_data

```

```

except Exception as e:
    print(f"❌ 爬取失败: {url} - {e}")
    return None

def extract_title(markdown: str) -> str:
    """从Markdown 中提取标题"""
    lines = markdown.split("\n")
    for line in lines:
        if line.startswith("# "):
            return line.replace("# ", "").strip()
    return "未知标题"

def extract_author(markdown: str) -> str:
    """从Markdown 中提取作者"""
    # 实现提取逻辑
    # 通常在 "By " 或 "作者: " 之后
    import re
    match = re.search(r"By\s+([A-Za-z\s]+)", markdown)
    if match:
        return match.group(1).strip()
    return "未知作者"

def extract_date(markdown: str) -> str:
    """从Markdown 中提取发布时间"""
    # 实现提取逻辑
    import re
    # 查找日期模式
    match = re.search(r"(\d{4}-\d{2}-\d{2})|\w+ \d{1,2}, \d{4})", markdown)
    if match:
        return match.group(1)
    return datetime.now().strftime("%Y-%m-%d")

# 测试
if __name__ == "__main__":
    test_url = "https://www.hawaiinewsnow.com/2025/01/27/some-news/"
    news = scrape_single_news(test_url)

    if news:
        print(f"✅ 成功爬取: {news['title']}")
        print(f"作者: {news['author']}")
        print(f"时间: {news['published_date']}")

```

```
# 保存为 JSON
```

```
with open(f"news_{datetime.now().strftime('%Y%m%d_%H%M%S')}.json", "w", encoding="utf-8") as f:  
    json.dump(news, f, ensure_ascii=False, indent=2)
```

步骤 3: 爬取新闻列表

```
def scrape_news_list(homepage_url: str, limit: int = 20) -> list:
    """
    爬取新闻列表

    Args:
        homepage_url: 首页 URL
        limit: 爬取数量限制

    Returns:
        新闻列表
    """
    try:
        # 使用 Crawl API
        result = app.crawl(
            url=homepage_url,
            limit=limit,
            scrape_options={
                "formats": ["markdown"],
                "only_main_content": True,
            },
            include_paths=["/2025/", "/news/"], # 只爬取新闻路径
            exclude_paths=["/weather/", "/sports/"], # 排除不需要的路径
        )

        news_list = []
        for item in result:
            if isinstance(item, tuple):
                success, data = item
                if success:
                    news_list.append({
                        "url": data.url,
                        "title": extract_title(data.markdown),
                        "author": extract_author(data.markdown),
                        "published_date": extract_date(data.markdown),
                        "content": data.markdown,
                    })
            else:
                news_list.append({
                    "url": item.url,
                    "title": extract_title(item.markdown),

```

```

        "author": extract_author(item.markdown),
        "published_date": extract_date(item.markdown),
        "content": item.markdown,
    })

    return news_list

except Exception as e:
    print(f"❌ 爬取失败: {e}")
    return []

# 测试
if __name__ == "__main__":
    homepage = "https://www.hawaiinewsnow.com/"
    news_list = scrape_news_list(homepage, limit=10)

    print(f"✅ 成功爬取 {len(news_list)} 篇新闻")

# 保存为 JSON
with open(f"news_list_{datetime.now().strftime('%Y%m%d')}.json", "w", encoding="utf-8") as f:
    json.dump(news_list, f, ensure_ascii=False, indent=2)

```

步骤 4: 使用 **Extract API** 结构化提取

```

from pydantic import BaseModel, Field
from typing import List, Optional

class NewsArticle(BaseModel):
    """新闻文章数据模型"""
    title: str = Field(..., description="新闻标题")
    author: str = Field(..., description="作者姓名")
    published_date: str = Field(..., description="发布日期")
    category: str = Field(..., description="新闻类别")
    summary: str = Field(..., description="新闻摘要")
    content: str = Field(..., description="新闻正文")
    tags: List[str] = Field(default_factory=list, description="标签列表")

def scrape_with_extract(url: str) -> Optional[NewsArticle]:
    """
    使用 Extract API 结构化提取新闻

    Args:
        url: 新闻 URL

    Returns:
        结构化新闻数据
    """
    try:
        result = app.scrape(
            url=url,
            formats=[{
                "type": "json",
                "schema": NewsArticle.model_json_schema(),
            }],
        )

        # 将 JSON 转换为 Pydantic 模型
        news = NewsArticle(**result.json)
        return news

    except Exception as e:
        print(f"✗ 提取失败: {url} - {e}")
        return None

```



```
# 测试

if __name__ == "__main__":
    test_url = "https://www.hawaiinewsnow.com/2025/01/27/some-news/"
    news = scrape_with_extract(test_url)

    if news:
        print(f"✅ 成功提取:")
        print(f"标题: {news.title}")
        print(f"作者: {news.author}")
        print(f"类别: {news.category}")
        print(f"摘要: {news.summary}")
        print(f"标签: {' '.join(news.tags)}")
```

步骤 5: 定时任务

```

import schedule
import time

def daily_news_scraper():
    """
    每日新闻爬取任务
    """
    print(f"🕒 {datetime.now()}: 开始爬取每日新闻...")

    # 爬取多个新闻源
    sources = [
        "https://www.hawaiinewsnow.com/",
        "https://www.staradvertiser.com/",
        "https://www.civilbeat.org/",
    ]

    all_news = []
    for source in sources:
        print(f"📰 爬取: {source}")
        news_list = scrape_news_list(source, limit=10)
        all_news.extend(news_list)

    # 保存数据
    filename = f"daily_news_{datetime.now().strftime('%Y%m%d')}.json"
    with open(filename, "w", encoding="utf-8") as f:
        json.dump(all_news, f, ensure_ascii=False, indent=2)

    print(f"✅ 完成! 共爬取 {len(all_news)} 篇新闻, 保存到 {filename}")

# 设置定时任务
schedule.every().day.at("08:00").do(daily_news_scraper) # 每天早上 8 点执行

print(f"🚀 新闻爬取定时任务已启动...")
print("每天 08:00 自动爬取夏威夷新闻")

# 运行定时任务
while True:
    schedule.run_pending()
    time.sleep(60) # 每分钟检查一次

```

学习建议

循序渐进学习路径

第 1 周：基础案例

- Day 1-2: 案例 01 (新闻采集)
- Day 3-4: 案例 04 (招聘信息)
- Day 5-6: 案例 05 (活动信息)
- Day 7: 总结和复习

第 2 周：进阶案例

- Day 1-2: 案例 02 (商家采集)
- Day 3-4: 案例 03 (租房监控)
- Day 5-6: 案例 06 (评论分析)
- Day 7: 实战项目

第 3 周：高级案例

- Day 1-2: 案例 11 (智能问答)
- Day 3-4: 案例 13 (内容生成)
- Day 5-7: 案例 14 (完整系统)

第 4 周：生产部署

- Day 1-5: 案例 15 (数据平台)
- Day 6-7: 优化和部署

实战技巧

1. **从简单开始** - 先掌握 Scrape API, 再学习 Crawl
 2. **多练习** - 每个案例至少运行 3 次
 3. **修改参数** - 尝试不同的配置选项
 4. **错误处理** - 添加完整的异常处理
 5. **数据验证** - 使用 Pydantic 验证数据
 6. **性能优化** - 使用缓存和批量处理
 7. **成本控制** - 监控 API 使用量
-

项目参考

Firecrawl 官方示例 (56 个)

位置: `/Users/zhiledeng/Downloads/FireShot/Firecrawl学习手册/05-实战案例/firecrawl-main-repo/examples/`

推荐学习顺序:

1. 爬虫基础 (学习 Crawl API)
 - `gpt-4.1-web-crawler`
 - `claude3.7-web-crawler`
 - `gemini-2.5-crawler`
 - `deepseek-v3-crawler`
2. 数据提取 (学习 Extract API)
 - `simple_web_data_extraction_with_claude`
 - `web_data_extraction`
 - `attributes-extraction-python-sdk.py`
3. 公司研究 (商家采集参考)
 - `gpt-4.1-company-researcher`
 - `deepseek-v3-company-researcher`
 - `R1_company_researcher`
4. 实战项目
 - `hacker_news_scraper`
 - `blog-articles`
 - `scrape_and_analyze_airbnb_data_e2b`
 - `deep-research-apartment-finder`
5. AI 集成
 - `web_data_rag_with_llama3`
 - `website_qa_with_gemini_caching`
 - `openai_swarm_firecrawl`

Firecrawl App Examples (40 个)

位置: `/Users/zhiledeng/Downloads/FireShot/Firecrawl学习手册/05-实战案例/示例应用/firecrawl-app-examples/`

HawaiiHub 推荐:

1. `company-data-scraper` - 商家数据采集
 2. `automated_price_tracking` - 价格监控
 3. `review-analyzer` - 评论分析
 4. `local-website-chatbot` - 智能问答
 5. `ai-resume-job-matching` - 招聘匹配
-

相关资源

文档

- 完整学习手册: `01-基础入门/Firecrawl完整学习手册.md`
- API 规范: `03-API参考/云端API规范.md`
- 配置指南: `04-配置指南/云端配置指南.md`
- 项目索引: `05-实战案例/示例项目完整索引.md`

官方资源

- 官方文档: <https://docs.firecrawl.dev/>
 - GitHub 主仓库: <https://github.com/firecrawl/firecrawl>
 - 示例仓库: <https://github.com/firecrawl/firecrawl-app-examples>
 - Discord 社区: <https://discord.gg/firecrawl>
-



下一步

准备好开始了吗?

推荐起点:

```
# 1. 创建项目目录
mkdir -p hawaiihub-firecrawl-cases
cd hawaiihub-firecrawl-cases

# 2. 创建第一个案例
mkdir case-01-news-scraper
cd case-01-news-scraper

# 3. 复制上面的代码，开始实践！
```

学习顺序:

1. ☒ 从案例 01 开始
2. ☒ 每完成一个案例，记录笔记
3. ☒ 遇到问题查阅文档
4. ☒ 参考官方示例项目
5. ☒ 最终构建完整系统

祝学习愉快，项目成功！ 🚀🌸

创建时间: 2025-10-27

版本: v1.0

维护者: HawaiiHub AI Team

适用项目: HawaiiHub - 夏威夷华人社区平台

Firecrawl 实战案例架构图集

创建时间: 2025-10-27

用途: 为PDF电子书提供可视化架构图和流程图

技术: Mermaid图表

案例 01: 夏威夷新闻采集器架构图

系统架构

```
graph TD
    A[用户请求] --> B[新闻采集器]
    B --> C{选择采集方式}
    C -->|单篇新闻| D[Scrape API]
    C -->|批量新闻| E[Crawl API]
    C -->|搜索新闻| F[Search API]
    D --> G[内容解析器]
    E --> G
    F --> G
    G --> H{数据验证}
    H -->|有效| I[数据存储]
    H -->|无效| J[错误日志]
    I --> K[JSON文件]
    I --> L[Markdown文件]
    I --> M[CSV文件]
    K --> N[数据分析]
    L --> O[人工审核]
    M --> P[Excel导入]
```

```
style A fill:#e1f5ff
style B fill:#fff3e0
style G fill:#f3e5f5
style I fill:#e8f5e9
```

数据流程

```
sequenceDiagram
    participant U as 用户
    participant S as 爬虫脚本
    participant F as Firecrawl API
    participant D as 数据库
    participant C as 缓存

    U->>S: 启动采集任务
    S->>C: 检查缓存

    alt 缓存命中
        C-->>S: 返回缓存数据
    else 缓存未命中
        S->>F: 发送爬取请求
        F->>F: 渲染页面 (JS支持)
        F->>F: 提取内容
        F-->>S: 返回Markdown数据
        S->>C: 更新缓存 (1小时)
    end

    end

    S->>S: 数据清洗
    S->>S: 格式化
    S->>D: 保存到数据库
    S-->>U: 返回采集结果
```

案例 02: 华人商家信息采集架构

系统架构

```
graph LR
    A【商家目录页】 --> B[Batch Scrape API]
    B --> C[商家详情页 x N]

    C --> D{提取信息}
    D --> E[基本信息]
    D --> F[联系方式]
    D --> G[营业时间]
    D --> H[用户评论]

    E --> I[商家数据库]
    F --> I
    G --> I
    H --> J[情感分析]

    J --> K[评分计算]
    K --> I

    I --> L[API服务]
    L --> M[HawaiiHub前端]

    style A fill:#fff3e0
    style B fill:#e1f5ff
    style I fill:#e8f5e9
    style M fill:#f3e5f5
```

数据模型

```
erDiagram
```

```

    RESTAURANT ||--o{ REVIEW : has
    RESTAURANT ||--o{ MENU_ITEM : offers
    RESTAURANT }o--|| CATEGORY : belongs_to
    RESTAURANT }o--|| LOCATION : located_at

```

```

    RESTAURANT {
        string id PK
        string name
        string address
        string phone
        string website
        float rating
        json hours
        datetime created_at
    }

```

```

    REVIEW {
        string id PK
        string restaurant_id FK
        string author
        int rating
        string content
        datetime date
    }

```

```

    MENU_ITEM {
        string id PK
        string restaurant_id FK
        string name
        float price
        string description
    }

```

```

    CATEGORY {
        string id PK
        string name
        string description
    }

```

```
LOCATION {  
  string id PK  
  string area  
  float latitude  
  float longitude  
}
```



案例 03: 租房信息监控系统架构

实时监控流程

```
graph TD
    A[定时任务 Cron] --> B[租房网站爬虫]
    B --> C[Firecrawl Crawl API]

    C --> D[新房源列表]
    D --> E[对比历史数据]

    E -->|新房源| F[标记为NEW]
    E -->|价格变动| G[标记为PRICE_CHANGE]
    E -->|已下架| H[标记为REMOVED]
    E -->|无变化| I[更新时间戳]

    F --> J[发送通知]
    G --> J
    H --> K[归档数据]

    J --> L[邮件通知]
    J --> M[Telegram Bot]
    J --> N[HawaiiHub App推送]

    I --> O[数据库更新]
    F --> O
    G --> O

    style A fill:#fff3e0
    style J fill:#ffebee
    style O fill:#e8f5e9
```

Change Tracking机制

sequenceDiagram

participant C as Cron任务

participant S as 爬虫服务

participant F as Firecrawl API

participant D as 数据库

participant N as 通知服务

Loop 每小时执行

C->>S: 触发爬取任务

S->>F: 请求爬取 (enableChangeTracking=**true**)

F->>F: 爬取页面

F->>F: 与上次快照对比

F-->>S: 返回变更内容

alt 有新内容

S->>D: 查询历史记录

D-->>S: 返回对比结果

S->>S: 计算价格变动

S->>N: 发送通知

N->>N: 用户A: 新房源提醒

N->>N: 用户B: 降价提醒

end

S->>D: 更新数据库

end

案例 11: 智能问答系统架构 (RAG)

RAG系统架构

```
graph TB
    A[用户问题] --> B[问题预处理]
    B --> C[向量化]
    C --> D[向量数据库 Pinecone]
    D --> E[检索Top-K相关文档]
    E --> F[重排序 Reranking]
    F --> G[构建Prompt]
    A --> G
    G --> H[LLM GPT-4]
    H --> I[生成答案]
    I --> J{答案质量检查}
    J -->|优质| K[返回用户]
    J -->|需改进| L[反馈循环]
    L --> M[Fine-tuning数据]
    M --> H

    subgraph 数据源
        N[新闻文章] --> O[Firecrawl采集]
        O --> P[商家信息]
        P --> Q[社区帖子]
        Q --> R[文档切片]
        R --> S[向量化]
        S --> D
    end

    style A fill:#e1f5ff
    style H fill:#fff3e0
    style D fill:#f3e5f5
    style K fill:#e8f5e9
```

数据处理流程

flowchart LR

A[原始网页] --> B[Firecrawl Scrape]

B --> C[Markdown内容]

C --> D{文档类型}

D -->|新闻| E[新闻解析器]

D -->|商家| F[商家解析器]

D -->|其他| G[通用解析器]

E --> H[文档切片 512 tokens]

F --> H

G --> H

H --> I[生成Embedding]

I --> J[Pinecone存储]

J --> K[索引优化]

K --> L[RAG检索]

style B fill:#fff3e0

style I fill:#e1f5ff

style J fill:#f3e5f5



案例 15: HawaiiHub完整数据平台架构

微服务架构

```
graph TB
    subgraph 用户层
        A[Web前端 Next.js]
        B[移动端 React Native]
        C[管理后台]
    end

    subgraph API网关
        D[Nginx负载均衡]
        E[API Gateway]
    end

    subgraph 服务层
        F[用户服务]
        G[商家服务]
        H[新闻服务]
        I[租房服务]
        J[搜索服务]
        K[推荐服务]
    end

    subgraph 数据采集层
        L[新闻爬虫]
        M[商家爬虫]
        N[租房爬虫]
        O[Firecrawl API]
    end

    subgraph 数据存储
        P[(PostgreSQL 主库)]
        Q[(Redis 缓存)]
        R[(Elasticsearch 搜索)]
        S[(Pinecone 向量)]
    end

    A --> D
    B --> D
    C --> D

    D --> E
```

E --> F
E --> G
E --> H
E --> I
E --> J
E --> K

L --> O
M --> O
N --> O

O --> P
F --> P
G --> P
H --> P
I --> P

F --> Q
G --> Q
H --> Q

J --> R
K --> S

style A fill:#e1f5ff
style O fill:#fff3e0
style P fill:#e8f5e9
style S fill:#f3e5f5

数据流转

```
sequenceDiagram
    participant U as 用户
    participant W as Web前端
    participant G as API Gateway
    participant S as 商家服务
    participant C as 爬虫服务
    participant F as Firecrawl
    participant D as 数据库
    participant R as Redis

    U->>W: 搜索"火奴鲁鲁中餐"
    W->>G: GET /api/restaurants?q=中餐
    G->>S: 转发请求

    S->>R: 检查缓存
    alt 缓存命中
        R-->>S: 返回缓存结果
    else 缓存未命中
        S->>D: 查询数据库
        D-->>S: 返回结果
        S->>R: 更新缓存 (10分钟)
    end

    S-->>G: 返回商家列表
    G-->>W: 返回JSON
    W-->>U: 展示结果

    Note over C,F: 后台定时任务 (每日凌晨2点)
    C->>F: 批量爬取商家信息
    F-->>C: 返回Markdown数据
    C->>C: 数据清洗和验证
    C->>D: 更新数据库
    C->>R: 清空相关缓存
```



技术栈总览

```
graph LR
  subgraph 前端
    A[Next.js 14]
    B[React Native]
    C[TailwindCSS]
  end

  subgraph 后端
    D[Python 3.11]
    E[FastAPI]
    F[Celery]
  end

  subgraph 数据采集
    G[Firecrawl API]
    H[Playwright]
  end

  subgraph 数据存储
    I[PostgreSQL]
    J[Redis]
    K[Elasticsearch]
    L[Pinecone]
  end

  subgraph AI/ML
    M[GPT-4]
    N[Gemini Pro]
    O[OpenAI Embeddings]
  end

  subgraph 部署
    P[Vercel]
    Q[Railway]
    R[Docker]
  end

  A --> E
  B --> E
  E --> D
```


D --> G

D --> F

E --> I

E --> J

E --> K

E --> M

E --> N

E --> O

O --> L

A --> P

E --> Q

F --> R

性能指标

系统性能对比

```
gantt
  title Firecrawl vs 传统爬虫性能对比
  dateFormat X
  axisFormat %s

  section 响应时间
  传统爬虫      :0, 5
  Firecrawl    :0, 1

  section 成功率
  传统爬虫      :0, 7
  Firecrawl    :0, 9.5

  section 开发时间
  传统爬虫      :0, 10
  Firecrawl    :0, 2
```

成本分析

方案	开发成本	运维成本	月度API费用	总成本（首年）
传统爬虫	\$5,000	\$800/月	\$0	\$14,600
Firecrawl	\$500	\$100/月	\$200/月	\$4,100
节省	-90%	-87.5%	+\$200	-71.9%



数据增长趋势

```
xychart-beta
```

```
  title "HawaiiHub 数据增长 (2025年预测)"
```

```
  x-axis [1月, 2月, 3月, 4月, 5月, 6月, 7月, 8月, 9月, 10月, 11月, 12月]
```

```
  y-axis "数据量 (千条)" 0 --> 100
```

```
  line [5, 8, 12, 18, 25, 35, 45, 58, 70, 82, 92, 100]
```

```
  line [2, 5, 8, 12, 18, 25, 33, 42, 52, 63, 75, 88]
```

说明:

- 蓝线: 新闻文章数量
- 橙线: 商家信息数量

最佳实践总结

开发流程

flowchart TD

A[需求分析] --> B{选择Firecrawl端点}

B -->|单页| C[Scrape API]

B -->|多页| D[Crawl API]

B -->|搜索| E[Search API]

C --> F[编写采集脚本]

D --> F

E --> F

F --> G[数据验证]

G --> H{质量检查}

H -->|通过| I[生产部署]

H -->|失败| J[优化参数]

J --> F

I --> K[监控告警]

K --> L{运行状态}

L -->|正常| M[持续运行]

L -->|异常| N[故障排查]

N --> O[修复问题]

O --> I

style A fill:#e1f5ff

style I fill:#e8f5e9

style K fill:#fff3e0

文档版本: v1.0

最后更新: 2025-10-27

维护团队: HawaiiHub AI Team

Firecrawl 完整项目总索引

更新时间: 2025-10-27
项目总数: 96 个 (40 + 56)
来源: GitHub 官方仓库

总览

仓库	项目数	路径	说明
firecrawl-app-examples	40	示例应用/firecrawl-app-examples/	完整应用示例
firecrawl/firecrawl (main)	56	firecrawl-main-repo/examples/	官方代码示例
总计	96	-	-

GitHub 仓库链接

- 1. 应用示例仓库: <https://github.com/firecrawl/firecrawl-app-examples>
- 2. 主仓库示例: <https://github.com/firecrawl/firecrawl/tree/main/examples>

🔥 仓库 1: firecrawl-app-examples (40 个完整应用)

仓库: <https://github.com/firecrawl/firecrawl-app-examples>

类型: 完整可运行应用

本地路径: 示例应用/`firecrawl-app-examples/`

分类清单

AI 智能应用（18 个）

项目	技术栈	功能描述
ai-resume-job-matching	Python	AI 简历职位匹配
claude-3.7-job-matcher	Python + Claude	Claude 3.7 职位匹配
deep-job-researcher	Next.js	深度职位研究
deepseek-v3-trend-finder	Node.js	DeepSeek 趋势分析
gemini-2-5-pro-trend-finder	Node.js	Gemini 趋势分析
gemini-2.5-trend-finder	Node.js	Gemini 2.5 趋势工具
mistral-small-3.1-trend-finder	Node.js	Mistral 趋势分析
post_predictor	Next.js	社交媒体热帖预测
deepseek-v3-agi-newsletter	Next.js	AGI 新闻简报
gemini-2-5-agi-newsletter	Next.js	Gemini 新闻简报
blog-thread-converter	Python	博客转推特线程
url-to-podcast	Next.js	URL 转播客
crewai_chatgpt_clone	Python + CrewAI	ChatGPT 克隆
local-website-chatbot	Next.js	本地网站聊天机器人
website-to-agent	Python + LangChain	网站转 AI Agent
deepseek-fine-tune	Python + Jupyter	DeepSeek 模型微调
gemma-custom-fine-tune	Python	Gemma 自定义微调
llama4-fine-tuning	Python	Llama 4 微调

数据采集与分析（8 个）

项目	技术栈	功能描述
change-detection-tutorial	Python	网站变更检测教程
automated_price_tracking	Python	自动化价格追踪
os-watch	Python	开源项目监控
company-data-scraper	Python	公司数据采集
review-analyzer	Python	评论分析
deepseek-rag	Python	DeepSeek RAG 知识库
mcp-document-reader	Python	MCP 文档阅读器
deep-research-endpoint	Python	深度研究端点

内容生成（7 个）

项目	技术栈	功能描述
search-to-report	Node.js	搜索转研究报告
search-to-slides	Node.js	搜索转演示文稿
search-to-mindmap	Node.js	搜索转思维导图
search-competitor-analysis	Node.js	竞品分析
search-email-to-company-intel	Node.js	邮件转公司情报
url-to-image-imagen4-gemini-flash	Next.js	URL 转图片
logo-tree-builder	Python	Logo 树状图生成

工具与优化（4 个）

项目	技术栈	功能描述
content-optimizer	Next.js	内容优化
hero-optimizer	Next.js	Hero 区块优化
roastmywebsite-example-app	Next.js	网站诊断
seo_generator_flow	Python	SEO 自动生成

教程与示例（3 个）

项目	技术栈	功能描述
google-adk-tutorial	Python	Google ADK 集成教程
change-detection-tutorial	Python	变更检测教程
custom-fine-tuning-dataset	Python	自定义微调数据集生成

 仓库 2: firecrawl/firecrawl (main)
(56 个代码示例)

仓库: <https://github.com/firecrawl/firecrawl>
路径: `/examples/`
类型: 代码示例和教程
本地路径: `firecrawl-main-repo/examples/`

分类清单

Web 爬虫 (24 个)

项目	AI 模型	说明
R1_web_crawler	R1	R1 模型驱动爬虫
claude3.7-web-crawler	Claude 3.7	Claude 3.7 爬虫
claude_stock_analyzer	Claude	股票分析爬虫
deepseek-v3-crawler	DeepSeek V3	DeepSeek 爬虫
gemini-2.0-crawler	Gemini 2.0	Gemini 2.0 爬虫
gemini-2.5-crawler	Gemini 2.5	Gemini 2.5 爬虫
gpt-4.1-web-crawler	GPT-4.1	GPT-4.1 爬虫
gpt-4.5-web-crawler	GPT-4.5	GPT-4.5 爬虫
grok_web_crawler	Grok	Grok 爬虫
groq_web_crawler	Groq	Groq 爬虫
haiku_web_crawler	Haiku	Haiku 爬虫
llama-4-maverick-web-crawler	Llama 4	Llama 4 Maverick 爬虫
mistral-small-3.1-crawler	Mistral 3.1	Mistral 爬虫
o1_web_crawler	O1	O1 爬虫
o3-mini_web_crawler	O3-mini	O3-mini 爬虫
o3-web-crawler	O3	O3 爬虫
o4-mini-web-crawler	O4-mini	O4-mini 爬虫
sonnet_web_crawler	Sonnet	Sonnet 爬虫
sales_web_crawler	-	销售线索爬虫
hacker_news_scraper	-	Hacker News 爬虫
blog-articles	-	博客文章爬虫

项目	AI 模型	说明
attributes-extraction-js-sdk.js	-	JavaScript 属性提取示例
attributes-extraction-python-sdk.py	-	Python 属性提取示例
web_data_extraction	-	通用数据提取

数据提取 (12 个)

项目	AI 模型	说明
claude3.7-web-extractor	Claude 3.7	Claude 提取器
gemini-2.0-web-extractor	Gemini 2.0	Gemini 提取器
gemini-2.5-web-extractor	Gemini 2.5	Gemini 提取器
llama-4-maverick-web-extractor	Llama 4	Llama 提取器
mistral-small-3.1-extractor	Mistral 3.1	Mistral 提取器
o1_web_extractor	O1	O1 提取器
simple_web_data_extraction_with_claude	Claude	Claude 简单提取
openai_swarm_firecrawl_web_extractor	OpenAI	OpenAI Swarm 提取
turning_docs_into_api_specs	-	文档转 API 规范
web_data_extraction	-	通用数据提取
attributes-extraction-js-sdk.js	-	JavaScript 提取
attributes-extraction-python-sdk.py	-	Python 提取

公司研究（6 个）

项目	AI 模型	说明
R1_company_researcher	R1	R1 公司研究
deepseek-v3-company-researcher	DeepSeek V3	DeepSeek 公司研究
gpt-4.1-company-researcher	GPT-4.1	GPT-4.1 公司研究
o3-mini_company_researcher	O3-mini	O3-mini 公司研究
crm_lead_enrichment	-	CRM 线索丰富
job-resource-analyzer	-	职位资源分析

分析工具（6 个）

项目	功能描述
claude-3.7-stock-analyzer	股票分析
scrape_and_analyze_airbnb_data_e2b	Airbnb 数据分析
visualize_website_topics_e2b	网站主题可视化
gemini-github-analyzer	GitHub 仓库分析
contradiction_testing	矛盾测试
job-resource-analyzer	职位资源分析

AI 应用 (5 个)

项目	功能描述
aginews-ai-newsletter	AI 新闻简报
ai-podcast-generator	AI 播客生成
o1_job_recommender	职位推荐系统
o3-mini-deal-finder	优惠发现
openai_swarm_firecrawl	OpenAI Swarm 集成

RAG 与问答 (2 个)

项目	技术栈
web_data_rag_with_llama3	Llama 3 RAG
website_qa_with_gemini_caching	Gemini 缓存

其他工具 (5 个)

项目	功能描述
find_internal_link_opportunities	内部链接发现
internal_link_assistant	内部链接助手
deep-research-apartment-finder	公寓深度研究
gemini-2.5-screenshot-editor	截图编辑器
openai-realtime-firecrawl	实时 API 集成

完整应用与部署（2 个）

项目	说明
full_example_apps	完整示例应用集
kubernetes	K8s 部署配置

 HawaiiHub 推荐项目（Top 20）

优先级 P0（立即使用）

排名	项目	仓库	应用场景
1	company-data-scraper	App	✅ 华人商家信息采集
2	automated_price_tracking	App	✅ 租房价格监控
3	review-analyzer	App	✅ 商家评论分析
4	hacker_news_scraper	Main	✅ 新闻爬取参考
5	blog-articles	Main	✅ 博客文章爬取

优先级 P1（近期集成）

排名	项目	仓库	应用场景
6	local-website-chatbot	App	✅ HawaiiHub 智能客服
7	deepseek-rag	App	✅ 知识库问答
8	web_data_rag_with_llama3	Main	✅ RAG 实现参考
9	gpt-4.1-company-researcher	Main	✅ 商家深度研究
10	deep-research-apartment-finder	Main	✅ 租房深度研究

优先级 P2（长期规划）

排名	项目	仓库	应用场景
11	deepseek-v3-trend-finder	App	✔ 热门话题发现
12	search-to-report	App	✔ 市场研究报告
13	content-optimizer	App	✔ 内容质量优化
14	seo_generator_flow	App	✔ SEO 自动优化
15	aginews-ai-newsletter	Main	✔ 社区简报生成
16	crm_lead_enrichment	Main	✔ 商家信息丰富
17	scrape_and_analyze_airbnb_data_e2b	Main	✔ 租房数据分析参考
18	full_example_apps	Main	✔ 完整应用架构参考
19	kubernetes	Main	✔ 生产部署参考
20	ai-podcast-generator	Main	✔ 内容生成参考

 技术栈统计

编程语言（96 个项目）

语言	项目数	占比
Python	45	47%
JavaScript/TypeScript	42	44%
其他（配置文件等）	9	9%

AI 模型分布

AI 模型	项目数	占比
GPT 系列	12	12.5%
Claude 系列	8	8.3%
Gemini 系列	11	11.5%
DeepSeek	7	7.3%
Llama 系列	6	6.2%
其他模型	8	8.3%
不使用 AI	44	45.8%

前端框架

框架	项目数	占比
Next.js	18	18.8%
React	12	12.5%
纯 Python	45	46.9%
其他	21	21.8%

本地路径

仓库 1: firecrawl-app-examples

```
# 路径
/Users/zhiledeng/Downloads/FireShot/Firecrawl学习手册/05-实战案例/示例应用/firecrawl-app-examples/

# 进入
cd "/Users/zhiledeng/Downloads/FireShot/Firecrawl学习手册/05-实战案例/示例应用/firecrawl-app-examp

# 查看项目
ls -la
```

仓库 2: firecrawl/firecrawl (main)

```
# 路径
/Users/zhiledeng/Downloads/FireShot/Firecrawl学习手册/05-实战案例/firecrawl-main-repo/examples/

# 进入
cd "/Users/zhiledeng/Downloads/FireShot/Firecrawl学习手册/05-实战案例/firecrawl-main-repo/exampL

# 查看项目
ls -la
```

同步更新

更新仓库 1

```
cd "/Users/zhiledeng/Downloads/FireShot/Firecrawl学习手册/05-实战案例/示例应用/firecrawl-app-examp
git pull origin main
```

更新仓库 2

```
cd "/Users/zhiledeng/Downloads/FireShot/Firecrawl学习手册/05-实战案例/firecrawl-main-repo/"
git pull origin main
```

相关文档

- 示例项目索引（仓库 1）：[示例项目完整索引.md](#)
- **HawaiiHub 实战手册**: [HawaiiHub实战案例手册.md](#)
- 学习手册: [../01-基础入门/Firecrawl完整学习手册.md](#)
- 学习路线图: [../00-手册导读/完整学习路线图.md](#)

总结

恭喜！你现在拥有 **96 个 Firecrawl 实战项目**：

-  40 个完整可运行应用
-  56 个官方代码示例
-  覆盖所有主流 AI 模型
-  涵盖爬虫、提取、分析、AI 等全场景
-  为 HawaiiHub 提供 20 个优先推荐项目

下一步：

1. 浏览 [HawaiiHub实战案例手册.md](#) 开始学习
2. 选择 Top 20 项目深入研究
3. 按学习路线图系统学习
4. 构建 HawaiiHub 数据采集系统

更新时间: 2025-10-27

总项目数: 96

维护团队: HawaiiHub AI Team

Firecrawl 版本: v2.4.0

Firecrawl 更新日志汇总

来源: <https://www.firecrawl.dev/changelog>
整理时间: 2025-10-27
版本跨度: v1.0.0 (2024-08) → v2.4.0 (2025-10)

版本演进时间线

v1.0.0 (2024-08-29) → v1.15.0 (2025-07-18) → v2.0.0 (2025-08-19) → v2.4.0 (2025-10-13)

└─ 首个稳定版 └─ v1 最终版 └─ 重大升级 └─ 最新版本

重大版本更新

v2.4.0 (2025-10-13) - 最新版本

新功能:

- **PDF 搜索分类:** 通过 `categories=["pdf"]` 专门搜索 PDF 文档
- **10倍语义爬取改进:** 使用 `prompt` 参数时准确性和相关性大幅提升
- **x402 搜索端点:** 通过 Coinbase x402 集成访问搜索 API
- **增强爬取状态:** robots.txt 限制和低结果场景的实时反馈
- **20+ 自部署改进:** 稳定性和功能大幅升级

v2.3.0 (2025-09-19)

新功能:

- **YouTube 转录支持:** 直接提取 YouTube 视频字幕

- **ODT & RTF 解析**: 新增文档格式支持
- **Docx 解析提速 50倍**: 性能大幅优化
- **企业自动充值**: 企业客户自动充值功能

v2.2.0 (2025-09-12)

新功能:

- **MCP v3**: HTTP Transport 和 SSE 模式稳定支持
- **Webhooks 增强**: 签名 + extract 支持 + 事件失败处理
- **Map 提速 15倍**: 支持更多 URL
- **新增地理位置**: CA、CZ、IL、IN、IT、PL、PT
- **API 密钥用量追踪**: 按密钥跟踪使用情况

v2.1.0 (2025-08-29)

新功能:

- **搜索分类**:
- `categories=["github"]` - GitHub 仓库、代码、Issues、文档
- `categories=["research"]` - 学术网站 (arXiv、Nature、IEEE、PubMed)
- **图片提取**: v2 scrape 端点支持图片提取
- **Data 属性爬取**: 支持提取 `data-*` 属性
- **Hash 路由**: Crawl 端点处理基于 hash 的路由
- **Google Drive 增强**: 支持 TXT、PDF、Sheets
- **Map 支持 100k 结果**: 大幅提升限制

v2.0.0 (2025-08-19) - 重大升级

核心改进:

- **默认更快**: 缓存默认 2 天, 默认启用 blockAds、skipTlsVerification、removeBase64Images
- **Summary 格式**: 新增 `formats=["summary"]` 直接获取页面摘要
- **JSON 提取更新**: 使用对象格式, 旧的 "extract" 改名为 "json"
- **增强截图选项**: 使用对象形式
- **新搜索源**: 支持 "news" 和 "images" 搜索
- **智能爬取**: 传入自然语言 prompt, 系统自动推导路径/限制

重大变化:

- **命名约定:** 驼峰式 → 下划线式 (`onlyMainContent` → `only_main_content`)
 - **返回类型:** 字典 → Document 对象 (`result["markdown"]` → `result.markdown`)
-

v1.15.0 (2025-07-18) - v1 最终版

- **SSO:** 企业级单点登录
- **FireGEO:** 开源地理数据示例
- 50+ PR 合并的 bug 修复和改进

v1.14.0 (2025-07-04)

- **认证爬取:** 支持需要登录的网站 (等待列表)
- **零数据保留:** 企业级数据隐私
- **MCP 改进:** maxAge + 更好的工具调用
- **Open Researcher:** 开源研究工具示例

v1.13.0 (2025-06-27)

- **Stealth Mode 扩展:** 新增 AU、FR、DE 地区
- **子域名爬取:** `allowSubdomains` 参数
- **Google Slides 爬取:** 官方支持
- **PDF 生成:** 生成当前页面的 PDF
- **Fireplexity:** 开源 Perplexity 示例

v1.12.0 (2025-06-20)

- **并发控制:** 请求级别的 max concurrency
- **整站爬取:** `crawlEntireDomain` 参数
- **Google Docs:** 官方支持
- **Firestarter:** 开源聊天机器人平台示例

v1.11.0 (2025-06-13)

- **Firecrawl Index:** 选择性开启后速度提升 500%
- **增强活动日志:** Webhook 事件 + 活跃爬取管理
- **Fire Enrich:** 开源 Clay 集成
- **Java SDK:** 社区 SDK 支持

v1.9.0 (2025-05-16)

- **自部署改进:** Supabase 修复 + LLM 提供商支持
- **MCP 改进:** 提示、示例和参数使用优化
- **Change Tracking:** SDK 2.0 新增变更追踪
- **Map 限制提升:** 5,000 → 30,000 链接

v1.6.0 (2025-03-07) - 关键里程碑

- **LLMs.txt API:** 将网站转换为 LLM-ready 文本
- **Deep Research API:** AI 驱动的深度研究 (Alpha)
- **官方 MCP Server:** Cursor/Windsurf/Claude 集成

v1.2.0 (2025-01-03)

- **Search API:** 搜索 + 爬取一体化
- MinerU PDF 解析成为默认

v1.1.0 (2024-12-27)

- 地理位置 + 移动爬取
- 批量爬取支持
- 4倍解析速度提升

v1.0.0 (2024-08-29) - 首个稳定版

- 输出格式选择
- Map 端点
- 2倍速率限制

- Go SDK + Rust SDK
- 团队支持
- API 密钥管理

v2 新功能亮点

1. 搜索分类系统

```
# GitHub 搜索
results = app.search(
    query="firecrawl python",
    categories=["github"]
)

# 学术搜索
results = app.search(
    query="AI machine learning",
    categories=["research"]
)

# PDF 搜索 (v2.4.0 新增)
results = app.search(
    query="研究报告",
    categories=["pdf"]
)
```

2. 智能提示爬取

```
# 传入自然语言描述
results = app.crawl(
    url="https://example.com",
    prompt="找到所有产品页面并提取价格信息"
)
```

3. Summary 格式

```
# 直接获取摘要
result = app.scrape(
    url="https://long-article.com",
    formats=["summary"]
)
print(result.summary)
```

4. YouTube 转录

```
# 提取视频字幕 (v2.3.0)
result = app.scrape(
    url="https://www.youtube.com/watch?v=xxx",
    formats=["markdown"]
)
```

 性能优化历程

版本	优化项	提升幅度
v1.11.0	Firecrawl Index	500% 速度提升
v2.2.0	Map 速度	15倍提升
v2.3.0	Docx 解析	50倍提升
v2.4.0	语义爬取	10倍准确性
v1.1.0	解析速度	4倍提升

SDK 版本对比

v1 vs v2 语法变化

功能	v1 语法	v2 语法
参数命名	<code>onlyMainContent=True</code>	<code>only_main_content=True</code>
缓存设置	<code>maxAge=3600000</code>	<code>max_age=3600000</code>
返回值访问	<code>result["markdown"]</code>	<code>result.markdown</code>
元数据访问	<code>result["metadata"]["title"]</code>	<code>result.metadata.title</code>

关键集成

MCP Server

- **v1.6.0** (2025-03-07): 首次官方支持
- **v1.14.0** (2025-07-04): maxAge + 工具调用改进
- **v2.2.0** (2025-09-12): v3 稳定版 (HTTP + SSE)

开源示例项目

- **FireGEO** (v1.15.0): 地理数据处理
- **Open Researcher** (v1.14.0): 研究助手
- **Fireplexity** (v1.13.0): Perplexity 克隆
- **Firestarter** (v1.12.0): 聊天机器人平台
- **Fire Enrich** (v1.11.0): 数据增强工具

文档资源

- 官方文档: <https://docs.firecrawl.dev/>
 - 更新日志: <https://www.firecrawl.dev/changelog>
 - **GitHub**: <https://github.com/firecrawl/firecrawl>
 - **Discord**: <https://discord.gg/firecrawl>
-

迁移建议

从 v1 迁移到 v2

1. 参数重命名 (全局替换)

```
bash # 批量替换 onlyMainContent → only_main_content maxAge → max_age  
blockAds → block_ads skipTlsVerification → skip_tls_verification
```

1. 返回值访问 (代码重构)

```
```python  
旧代码
content = result.get("markdown")
title = result.get("metadata", {}).get("title")

新代码
content = result.markdown
title = result.metadata.title
```
```

1. 测试验证

```
bash python3 setup_sdk.py # 验证 SDK v2 python3 test_api_keys.py # 测试  
API python3 quick_start.py # 运行示例
```

最后更新: 2025-10-27

整理人: HawaiiHub AI Team

数据来源: Firecrawl 官方 Changelog